



UNIVERSITY OF BAMBERG

Faculty of Information Systems and Applied Computer Sciences

BACHELOR'S THESIS

**A journey through Hashiwokakero:  
Analysis, algorithms and solving approaches  
for a complex puzzle**

BY

Hannah Rattenberger

*Supervisor:*

Prof. Dr. Isolde Adler

Chair for Algorithms and Complexity Theory

11.05.2026

## **Abstract**

Hashiwokakero is a Japanese logic puzzle in which islands must be connected with bridges while adhering to several rules. This thesis studies Hashiwokakero from both a theoretical and practical perspective. We formally define the puzzle, discuss its computational complexity, and develop three solvers. The first solver is based on SAT, the second on SMT, and the third solver uses a deduction-based strategy inspired by human reasoning. In addition, we develop a puzzle generator and evaluate all solvers on both a generated test suite and a test suite from the literature. While the deduction-based solver performs best on most instances, the SMT-based solver with reachability encoding is the most reliable approach and solves the most instances within the time bounds. The results show that performance is not only influenced by instance size, but also by structural puzzle properties.

# Contents

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                          | <b>1</b>  |
| <b>2</b> | <b>Background</b>                            | <b>2</b>  |
| 2.1      | Formal Definition . . . . .                  | 2         |
| 2.2      | Related Work . . . . .                       | 3         |
| <b>3</b> | <b>Computational Complexity</b>              | <b>5</b>  |
| 3.1      | Background . . . . .                         | 5         |
| 3.2      | Hashiwokakero is NP-Complete . . . . .       | 6         |
| 3.2.1    | Hashiwokakero is in NP . . . . .             | 6         |
| 3.2.2    | Hashiwokakero is NP-Hard . . . . .           | 10        |
| <b>4</b> | <b>Solving Hashiwokakero with SAT</b>        | <b>15</b> |
| 4.1      | SAT Problem . . . . .                        | 15        |
| 4.2      | Hashiwokakero as a SAT Problem . . . . .     | 18        |
| 4.2.1    | Rules 1 to 5 . . . . .                       | 19        |
| 4.2.2    | Rule 6: Connectivity . . . . .               | 21        |
| <b>5</b> | <b>Solving Hashiwokakero with SMT</b>        | <b>26</b> |
| 5.1      | SMT Problem . . . . .                        | 26        |
| 5.2      | Hashiwokakero as an SMT Problem . . . . .    | 27        |
| 5.2.1    | Rules 1 to 5 . . . . .                       | 27        |
| 5.2.2    | Rule 6: Connectivity . . . . .               | 28        |
| <b>6</b> | <b>Solving Hashiwokakero with Deductions</b> | <b>31</b> |
| 6.1      | Foundations . . . . .                        | 31        |
| 6.2      | Algorithm . . . . .                          | 33        |
| 6.3      | Deduction Rules . . . . .                    | 35        |
| <b>7</b> | <b>Evaluation</b>                            | <b>39</b> |
| 7.1      | Generator . . . . .                          | 39        |
| 7.2      | Benchmark Setup . . . . .                    | 41        |
| 7.3      | Results . . . . .                            | 42        |
| 7.3.1    | Puzzles with a Unique Solution . . . . .     | 43        |
| 7.3.2    | Puzzles with Multiple Solutions . . . . .    | 43        |
| 7.3.3    | Puzzles from Literature . . . . .            | 47        |
| 7.4      | Discussion . . . . .                         | 50        |
| <b>8</b> | <b>Conclusion</b>                            | <b>54</b> |

# 1 Introduction

Hashiwokakero is a logic puzzle invented by Nikoli, a Japanese publisher that popularised famous puzzles, such as Sudoku, Kakuro, Slitherlink, and Nurikabe. The name ‘Hashiwokakero’ translates as ‘Build Bridges’. In other parts of the world, it is also known as ‘Hashi’, ‘Bridges’, or ‘Ai-Ki-Ai’. Hashiwokakero belongs to the category of Japanese logic puzzles, which are characterised by simple rules, independence of language and culture, and a unique solution that can be derived using deduction [1].

Compared to other Japanese logic puzzles, the literature on Hashiwokakero is rather scarce. This makes Hashiwokakero an interesting subject for theoretical and practical research. The goal of this thesis is to analyse the computational complexity of Hashiwokakero and to develop and compare different solving approaches. In particular, the goals for the solving approaches are (i) to investigate how Hashiwokakero can be encoded as a SAT or SMT problem, as there exist highly optimised solvers for these problems, (ii) to develop a dedicated Hashiwokakero solving algorithm that uses human-like reasoning, and (iii) to evaluate the performance of the different solving approaches.

**Methodology** To analyse the computational complexity of Hashiwokakero, we establish the complexity class Hashiwokakero belongs to and present a proof from literature. For the SAT- and SMT-based approaches, we develop a constraint-based encoding from the rules of Hashiwokakero. For the deduction-based solver, we develop an algorithm and a set of deduction rules inspired by human solving approaches, complemented with a backtracking strategy. To be able to compare the performance of the approaches, we implement a puzzle generator and complement the generated test suite with an additional test suite from the literature.

**Structure** In Chapter 2, the rules for Hashiwokakero are introduced, the puzzle is formally defined, and related work is reviewed. In Chapter 3, the computational complexity of Hashiwokakero is analysed. In Chapters 4, 5 and 6, a SAT-based, an SMT-based, and a deduction-based solving approach to Hashiwokakero are presented. In Chapter 7, the performance of the solvers is evaluated and discussed, and Chapter 8 concludes the thesis.

## 2 Background

Hashiwokakero is defined by six rules.

1. Bridges must begin and end at distinct islands.
2. Bridges may only run horizontally or vertically.
3. Bridges must not cross any other bridges or islands.
4. At most two bridges may connect any pair of islands.
5. The number of bridges connected to each island must match the number indicated on that island.
6. All islands must be connected so that each island can be reached from any other island.

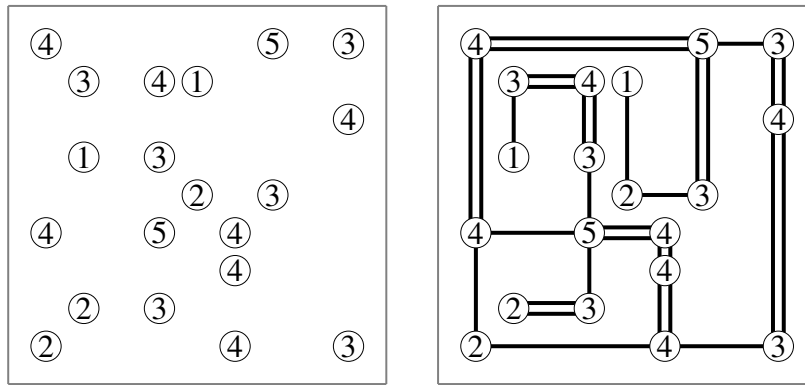


Figure 1: Example of a Hashiwokakero puzzle (left) and a valid solution (right).

Figure 1 illustrates an example of a valid solution. Having defined the rules, we now formalise the puzzle.

### 2.1 Formal Definition

Hashiwokakero is a puzzle consisting of *islands* and *bridges*. Islands are represented as circles labelled with numbers, and *bridges* are represented as straight line segments connecting these circles.

Islands are formally denoted using lowercase letters  $a, b, c, \dots$ . The set  $I = \{a, b, c, \dots\}$  contains all islands in the puzzle. The number written on each island specifies how many bridges must be connected to it. This number is called the *bridge constraint*. For each island  $a \in I$ , the bridge constraint is denoted by  $num(a) \in \{1, \dots, 8\}$ . The maximum number for a bridge constraint is eight, as no more than eight bridges can connect to a single island. The minimum number is one, as each island must be connected to at least one other island to avoid violating Rule 6.

Two islands are called *neighbours* or *neighbouring islands* when they are in the same row or the same column, and no other island lies in between them. For an island  $a \in I$ , the set of all its neighbours is denoted by  $neighbours(a)$ .

A bridge is identified by the pair of islands it connects. The two bridges between islands  $a, c \in I$  are denoted by  $B_{ac}^1$  and  $B_{ac}^2$ , where  $B_{ac}^1$  represents the first and  $B_{ac}^2$  the second bridge between these islands. The notations  $B_{ac}^i$  and  $B_{ca}^i$  denote the same bridge.

We say that a bridge is *set* if it is part of a solution, and *unset* if it is not part of a solution. If its status is not yet determined, we say the bridge is *undecided*. If a horizontal and a vertical bridge cross, we say they *collide*. A puzzle has the property of being *unique* if only one solution exists. Otherwise, the puzzle has *multiple solutions*.

## 2.2 Related Work

We start with reviewing the relevant literature around Hashiwokakero. Compared to other Japanese logic puzzles, the literature is rather scarce. We start with reviewing theoretical research, before we cover research on solving techniques.

Many puzzle games, including Sudoku, Kakuro, Nurikabe, and Slitherlink, among others [2], have been shown to be NP-complete. In 2009, Andersson proved that this also holds true for Hashiwokakero, via a reduction from the problem of Hamiltonian circuits in grid graphs [3].

The NP-completeness of Hashiwokakero is particularly relevant to this thesis, as it implies that Hashiwokakero can be reduced to other NP-complete problems, such as SAT and SMT, which allows us to implement solving approaches using SAT and SMT solvers. Further theoretical work has been conducted by Golan, who demonstrated that Hashiwokakero can be reduced to Minesweeper [4] and by Ruangwises and Itoh, who introduced a card-based zero-knowledge proof protocol, for which they used Hashiwokakero as an application [5]. Beyond theoretical research, several authors have focused on algorithms and practical solving approaches. Coelho et al. proposed a branch-and-cut algorithm and a mathematical programming model for Hashiwokakero [6]. They also implemented a puzzle generator and published a benchmark test suite with Hashiwokakero instances that we will use in this thesis to complement our own generated instances. Their work further analyses how generator parameters, such as the number of double bridges and cycles, influence the performance of their algorithm.

Other approaches are inspired by human-like solving techniques. Malik et al. combine heuristic operations with backtracking. Their heuristics resemble the deductions used by human players, while backtracking is employed when the heuristic operations are not applicable [7]. Similarly, the deduction-based solver presented as the third solving approach in this thesis applies deduction rules combined with a backtracking approach to emulate human-like solving strategies. Browne introduced deductive search, a breadth-first, constraint-based method for solving deduction puzzles, including Hashiwokakero [1]. This approach also emulates the strengths and weaknesses of human reasoning.

Constraint-based and declarative solving techniques have also been studied by several other authors. Brain et al. presented an ASP encoding for Hashiwokakero [8], while Prosser

and Unsworth investigated rooted tree and spanning tree constraints. They introduced a connectivity constraint and a no-cycle constraint, which they applied to Hashiwokakero [9]. In addition to solving approaches, Hashiwokakero has also been used as a benchmark problem. Lloyd et al. present a framework for studying combinatorial black-box optimisation, using Hashiwokakero as one of the applications [10]. In a recent paper from 2026, Maniparambil et al. explored the use of large language models for solving several topological grid puzzles, including Hashiwokakero [11]. Having reviewed the relevant literature, we now analyse the computational complexity of Hashiwokakero, building on the NP-completeness proof by Andersson [3].

## 3 Computational Complexity

In this chapter, we analyse the time complexity of Hashiwokakero. Even if a problem is theoretically solvable, this does not guarantee the existence of a practical solution that can be computed within reasonable time and memory limits [12]. A time complexity analysis thus helps to distinguish between problems that can be efficiently solved in practice and intractable problems that may become difficult to solve for large inputs [13].

### 3.1 Background

Complexity analysis classifies problems into different complexity classes. The two classes relevant to our context are P and NP.

#### Definition 3.1 (Class P [14])

$P$  is the class of languages that are decidable in polynomial time on a deterministic single-tape Turing machine.

This means that problems in  $P$  can be solved by a deterministic algorithm whose running time grows at most polynomially with the size of the problem input. Thus, many problems in  $P$  can be efficiently solved in practice. Problems in  $NP$  are those problems for which finding a solution might require more than polynomial time. A given solution to these problems, however, can be verified in polynomial time by a verifier, an algorithm that accepts an input if and only if there exists some solution that satisfies all problem constraints.

#### Definition 3.2 (Verifier [14])

A *verifier* for a language  $L$  is an algorithm  $A$ , such that

$$L = \{w \mid A \text{ accepts } \langle w, c \rangle \text{ for some string } c\}$$

The string  $c$  is called a *certificate* of membership in  $L$ .

A *polynomial-time verifier* is a verifier that runs in polynomial time in the length of  $w$ .

We can now use this concept of a polynomial-time verifier to define the class  $NP$ .

#### Definition 3.3 (Class NP [14])

$NP$  is the class of languages that have polynomial-time verifiers.

When discussing the complexity of a problem, it is helpful to consider the properties of NP-hardness and NP-completeness. A problem is NP-hard if it is at least as complex as any problem in  $NP$ , which means that finding a practical solution for large instances might be difficult. And it is NP-complete when it is just as complex as any other problem in  $NP$ .

#### Definition 3.4 (NP-Hardness)

A problem  $L$  is NP-hard if every problem in  $NP$  can be reduced to  $L$  in polynomial time.

#### Definition 3.5 (NP-Completeness)



A problem  $L'$  is NP-complete if  $L'$  is NP-hard and  $L' \in NP$ .

With this background, we return to Hashiwokakero and show that it is not only in NP but also NP-complete.

## 3.2 Hashiwokakero is NP-Complete

In 2009, Andersson proved that Hashiwokakero is NP-complete [3]. By Definition 3.5, Hashiwokakero has to satisfy two conditions to be NP-complete:

1. Hashiwokakero is in NP.
2. Hashiwokakero is NP-hard.

In the following, we demonstrate that Hashiwokakero satisfies both of these conditions.

### 3.2.1 Hashiwokakero is in NP

We first show that Hashiwokakero belongs to the class NP. This part is omitted in the proof by Andersson [3]. To do so, we will describe a polynomial-time verifier for the language of puzzle instances. This verifier accepts a puzzle input if there exists some correct solution as a certificate, and rejects it otherwise. By constructing this algorithm, we prove that Hashiwokakero is in NP. We begin by establishing the following lemma.

#### Lemma 3.1

There exists a polynomial-time verifier for Hashiwokakero.

*Proof.* We prove the lemma by describing a verification procedure for each rule of Hashiwokakero and showing that each verification step can be performed in polynomial time. The input to the verifier is a Hashiwokakero instance, encoded as a word row by row. Each symbol in the word is either 0, representing an empty cell, or a positive number, representing the bridge constraint of an island at that position. Rows are separated by the symbol #. Figure 2 illustrates this encoding.

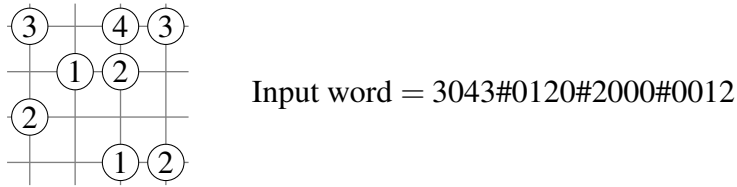


Figure 2: Example of a puzzle and the corresponding input word, illustrating the representation of a puzzle for the verifier.

A correct solution to the input instance serves as the *certificate*. This certificate is formed by a list of all bridges that are present in the solution. Each bridge is specified by the coordinates (row, column) of its two islands, separated by a point, and an id indicating whether it is the first (B1) or second (B2) bridge between these islands. As an example, the first bridge between islands at the positions (0, 2) and (0, 3) is denoted by B1.0.2.0.3 and

the second bridge between them by  $B2.0.2.0.3$ . The bridges are separated by the symbol #. Figure 3 illustrates this representation.

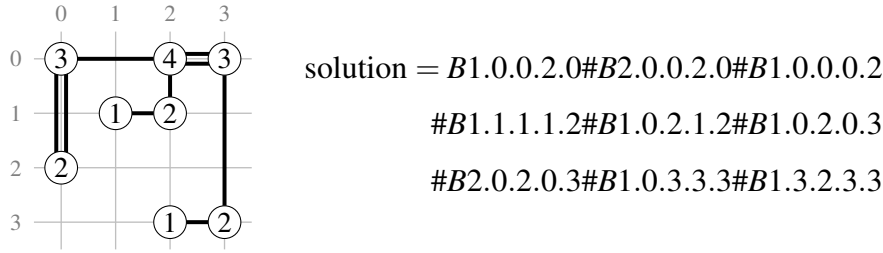


Figure 3: Example of a puzzle solution that serves as a certificate for the corresponding puzzle, illustrating the representation of a certificate for the verifier.

We will now describe an algorithm that accepts only words composed of puzzle input and a correct solution. The algorithm verifies each of the six Hashiwokakero rules in polynomial time, and rejects the solution if any rule is violated. Note that to prove Lemma 3.1, it is sufficient to find any algorithm that verifies the solution in polynomial time; the algorithm does not need to be efficient or optimal.

To estimate the time complexity of the rules, we do not need the exact numbers of islands and bridges. Instead, we need an upper bound that depends on the length of the input word representing the puzzle. As we need to refer to the upper bounds for islands and bridges multiple times throughout the algorithm, we establish these two upper bounds in the beginning to avoid repetition.

Let  $n$  be the length of the input puzzle. The number of islands,  $i$ , has  $n$  as an upper bound, as each island is represented in the input word. For the number of bridges  $b$  in the solution, we use  $2 * n^2$  as an upper bound. Each pair of islands can have at most two bridges, and there are fewer than  $i^2$  possible pairs of islands. Using  $i \leq n$  from above, we get  $b \leq 2 * i^2 \leq 2 * n^2$ . With these bounds established, we will now describe the verification procedure for each rule.

**Rule 1:** We need to ensure that each bridge starts and ends at distinct islands. For this, we go through the solution and compare each pair of bridge coordinates with the puzzle input to find out whether there is an island at this exact position. If an island can be found for each pair of coordinates in the solution and both coordinate pairs are distinct, this rule is satisfied.

Since there are at most  $2n^2$  bridge endpoints to check, and locating the corresponding position in the puzzle input of length  $n$  requires scanning up to  $n$  symbols, the total time complexity for verifying this rule is  $\mathcal{O}(2n^2 \cdot n) = \mathcal{O}(n^3)$ .

**Rule 2:** We need to verify that each bridge is either horizontal or vertical. To do this, we examine every bridge in the solution and check the coordinates of its two island endpoints. If either the row indices or the column indices of the two islands are the same, while the other indices differ, the bridge is aligned correctly, and the rule is satisfied.

Since there are at most  $2n^2$  bridges in the solution, we need to perform this check for each bridge. Comparing the row and column indices of a bridge's endpoints on a Turing machine requires linear time, so the total time complexity for verifying this rule is  $\mathcal{O}(2n^2 \cdot n) = \mathcal{O}(n^3)$ .

**Rule 3:** We need to verify that bridges in the solution do not collide with islands or other bridges. To do this, we first copy the puzzle input so that we can work on it without modifying the original. Then, we go through the solution. For any bridge, we check whether it is horizontal or vertical by comparing the indices. We then examine all fields the bridge covers, excluding the starting and ending islands. If a field is empty, we mark it with a  $-$  symbol for a horizontal bridge and with  $|$  for a vertical bridge in the puzzle copy. If a field we examine already contains a number, Rule 3 is violated as some bridge crosses an island. If a field already contains the symbol  $-$  while examining a vertical bridge or the symbol  $|$  while examining a horizontal bridge, then we found a collision of bridges and Rule 3 is also violated. If this process can be completed for every bridge without detecting a collision, the rule is satisfied.

Copying the puzzle requires scanning and copying each of the  $n$  elements in the input, which can be done in  $\mathcal{O}(n)$ . For the verification step, we examine each of the at most  $2n^2$  bridges in the solution and, within linear time, check its orientation. For each bridge, we then check all fields it may cover, which is at most the number of rows of the puzzle minus two for a vertical bridge, or the number of columns of the puzzle minus two for a horizontal bridge. Using  $n = r \cdot c$  as an upper bound for both  $r - 2$  and  $c - 2$ , this step has a time complexity of  $\mathcal{O}(2n^2 \cdot (n + n)) = \mathcal{O}(n^3)$ . Thus, the total time complexity for this rule is  $\mathcal{O}(n + n^3) = \mathcal{O}(n^3)$ .

**Rule 4:** We need to verify that between each pair of islands, at most two bridges are set. To do this, we first copy the solution so that we can work on it without modifying the original. Then, we go through this copy. For each bridge, we first store the coordinates of the two island endpoints and then go through all other bridges to count how many bridges connect this same pair of islands in total. While scanning the solution, we overwrite any appearance with  $*$  symbols so that it is not considered again. If any bridge appears more than twice, the solution is invalid; otherwise, this rule is satisfied.

Copying the solution requires scanning and duplicating each of the at most  $2n^2$  bridges. This can be done in  $\mathcal{O}(n^2)$ . Then, we do the verification step: For each bridge, we scan all remaining bridges to count occurrences between the same pair of islands. Since there are at most  $2n^2$  bridges to be scanned, this results in a time complexity of  $\mathcal{O}(n^4)$ . The total time complexity for this rule is thus  $\mathcal{O}(n^2 + n^4) = \mathcal{O}(n^4)$ .

**Rule 5:** We need to verify that the number of bridges connected to each island matches the bridge constraint of that island. To do this, we first create a copy of the puzzle. Then,

for each bridge, we take the coordinates of the two islands it connects, and for each of these islands, we decrement the number in the puzzle copy by one. After processing all bridges in the solution, we scan the puzzle copy to check the numbers on each island. If any number is different from 0, the solution is invalid; otherwise, this rule is satisfied.

Copying the puzzle requires scanning and duplicating each of the  $n$  elements, which takes  $\mathcal{O}(n)$  time. Then, we do the verification step: For each bridge, we need to access both islands it connects in the puzzle copy. Since we have at most  $2n^2$  bridges, and each access may require scanning up to  $n$  elements, this step takes  $\mathcal{O}(n^3)$  time. Finally, we scan the puzzle copy once to verify that all numbers are zero, which adds  $\mathcal{O}(n)$  time. Therefore, the total time complexity for this rule is  $\mathcal{O}(n + n^3 + n) = \mathcal{O}(n^3)$ .

**Rule 6:** We need to verify that all islands are connected. For this, we use a breadth-first search approach. First, we scan the puzzle input once to collect all islands and store their coordinates in a list. We then take the first island in this list as the starting point and add it to a new list for connected islands.

For each island in the connected list, we iterate through the solution to find all bridges connected to that island. For each such bridge, we add the second island it connects to the connected list if it is not already present. This process is repeated exactly once for each island in the connected islands list until there are no new islands left on the list. Finally, we compare the length of the connected list with the list of all islands. If the lengths are equal, all islands are connected, and the rule is satisfied.

Scanning the puzzle input once to record all islands takes  $\mathcal{O}(n)$  time. Then, for each island in the list of connected islands, we scan all bridges to find those connected to it, while adding the endpoints of these bridges to the list of connected islands. Since each of the at most  $n$  islands is added to the list once, or not at all, and there are at most  $2n^2$  bridges in the solution that might be considered at some point, this step has a time complexity of  $\mathcal{O}(n^3)$ . Comparing the lengths of the two lists takes  $\mathcal{O}(n)$  time. Therefore, the total time complexity for this rule is  $\mathcal{O}(n + n^3 + n) = \mathcal{O}(n^3)$ .

As we just demonstrated, each rule can be verified in polynomial time. Summing the individual complexities yields a total verifier time complexity of  $\mathcal{O}(n^3 + n^3 + n^3 + n^4 + n^3 + n^3) = \mathcal{O}(n^4)$ . Since this time complexity is polynomial, we have found a polynomial-time verifier for the puzzle Hashiwokakero.  $\square$

With Lemma 3.1, we have now established that Hashiwokakero has a polynomial-time verifier. By Definition 3.3, we therefore conclude the following theorem.

**Theorem 3.1 (Hashiwokakero is in NP)**

Hashiwokakero is in the complexity class NP.

This completes the proof that Hashiwokakero is in the class NP. We will now show that Hashiwokakero is NP-hard, thereby proving that it is also NP-complete.

### 3.2.2 Hashiwokakero is NP-Hard

As stated in Definition 3.4, a problem is NP-hard if any problem in NP can be reduced to it in polynomial time. Thus, if some NP-hard problem can be reduced to Hashiwokakero in polynomial time, then any other problem in NP can also be reduced to Hashiwokakero via two polynomial reductions. First, the problem is reduced to the NP-hard problem, and second, this NP-hard problem is reduced to Hashiwokakero. Since the sum of two polynomial functions is still polynomial, this would prove the NP-hardness of Hashiwokakero.

Using this approach, Andersson proved the NP-completeness of Hashiwokakero through a reduction from the problem of Hamiltonian circuits in grid graphs [3]. The latter problem was shown to be NP-complete in 1982 [15]. Since any problem in NP can be reduced to the NP-complete Hamiltonian circuit problem in polynomial time, finding a polynomial reduction from the Hamiltonian circuit problem to Hashiwokakero suffices to prove that Hashiwokakero is NP-hard. We follow Andersson's approach [3], but first, we introduce the Hamiltonian circuit problem in grid graphs. To define grid graphs, we first need to introduce graphs and unit distance graphs.

#### Definition 3.6 (Graph)

A graph  $G$  is a tuple  $(V, E)$ , where  $V$  is a finite set of vertices and  $E \subseteq \{\{x, y\} : x, y \in V\}$  is the set of edges between these vertices.

With this definition of a graph, we can now consider a unit distance graph, in which edges exist only between vertices that are directly adjacent.

#### Definition 3.7 (Unit distance graph [16])

A graph  $G = (V, E)$  is a unit distance graph if  $E = \{(x, y) : x, y \in V, |x - y| = 1\}$ , where  $|x - y|$  denotes the Euclidean distance between  $x$  and  $y$ .

This leads to the grid graph, a unit distance graph in which all vertices lie on a plane with integer coordinates.

#### Definition 3.8 (Grid graph [15])

A grid graph is a node-induced finite subgraph of the infinite grid.

Now that we have defined grid graphs, we turn to Hamiltonian circuits. We begin by defining paths, which leads to the concept of circuits and, finally, Hamiltonian circuits.

#### Definition 3.9 (Path)

In an undirected graph  $G = (V, E)$ , a path is a sequence of vertices  $(v_0, \dots, v_n) \subseteq V^n$ , with  $v_j \neq v_k$  for  $j \neq k$ , s.t.  $\{v_{i-1}, v_i\} \in E$  for  $i \in \{1, \dots, n\}$ .

Based on the definition of a path, we define a circuit as a path whose first and last vertices are the same.

#### Definition 3.10 (Circuit)

In an undirected graph  $G = (V, E)$ , a circuit is a path  $(v_0, \dots, v_n)$  with  $v_0 = v_n$ .

A Hamiltonian circuit additionally contains each vertex in a graph.

**Definition 3.11 (Hamiltonian Circuit)**

In an undirected graph  $G = (V, E)$ , a Hamiltonian circuit is a circuit that contains every vertex in  $V$  exactly once.

We now have all the necessary terminology to define the NP-complete problem of finding Hamiltonian circuits in grid graphs.

**Definition 3.12 (Hamiltonian Circuits in Grid Graphs Problem)**

Given a finite set  $P \subset \mathbb{Z}^2$ , is there a Hamiltonian circuit in the unit distance graph of  $P$ ?

We now present Andersson's reduction from the problem of Hamiltonian circuits in grid graphs to Hashiwokakero [3]. This reduction is used to prove the following lemma.

**Lemma 3.2**

The problem of Hamiltonian circuits in grid graphs can be reduced to Hashiwokakero in polynomial time.

*Proof.* Let  $P \subset \{(x, y) : x, y \in \mathbb{Z}\}$  be a set of points, and let  $G$  be the corresponding unit-distance graph. Figure 4 shows an example.

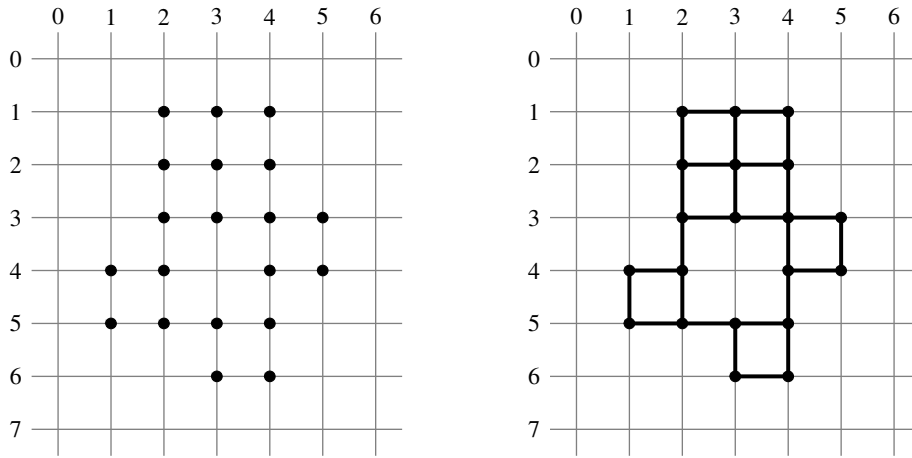


Figure 4: A set of points  $P$  (left) and the corresponding unit distance graph  $G$  (right).

Before applying the main reduction, small instances with  $|P| < 4$  are handled separately. This is necessary, as the proposed construction could yield a solvable Hashiwokakero instance for some small cases, even though no Hamiltonian circuit exists in the grid graph. Since a circuit in a grid graph requires at least four distinct vertices, no Hamiltonian circuits exist for  $|P| < 4$ . For these cases, we thus construct trivially unsolvable Hashiwokakero puzzles, like a single island with a bridge constraint of 1. Andersson included the case  $|P| = 3$  in the normal construction, which also works. However, since a Hamiltonian circuit on three points is impossible in a grid graph, this case can be treated like the other small instances by constructing an unsolvable puzzle. With the small cases excluded, we

can now assume that  $|P| \geq 4$  and proceed with constructing a Hashiwokakero instance corresponding to the grid graph of  $P$ .

Similar to the coordinate system for the grid graph of  $P$ , we introduce a second, initially empty coordinate system for the Hashiwokakero instance. For each point  $p = (x, y) \in P$ , we place a Hashiwokakero island at  $(2x, 2y)$ . These *big islands*, as Andersson calls them, correspond one-to-one to the points in  $P$ . We now add a bridge constraint for each big island, depending on the number of points at a unit distance from the corresponding point  $p$ . Thus, the big island at  $(2x, 2y)$ , that is constructed from the point  $p = (x, y)$ , receives a bridge constraint of  $6 - |\{q \in P : |p - q| = 1\}|$ . This means a point with no other points at unit distance receives a bridge constraint of 6, a point with one adjacent point receives 5, a point with two adjacent points receives 4, a point with three adjacent points receives 3, and a point that is fully surrounded by four other points at unit distance receives 2 as a bridge constraint.

Now that we have placed the big islands, we partially surround them with *small islands*. For each point  $(x, y)$ , we select from the four orthogonally surrounding grid positions at unit distance those that are not  $P$ , and call them  $(x', y')$ . Then, we place small islands at  $(x + x', y + y')$ . These small islands all have a bridge constraint of 1. This construction ensures that every big island is surrounded in all four directions. In each direction, there is either a big island at a distance of two or a small island at a distance of one. With this, we have completed the reduction as described by Andersson [3]. An example of this reduction is illustrated in Figure 5.

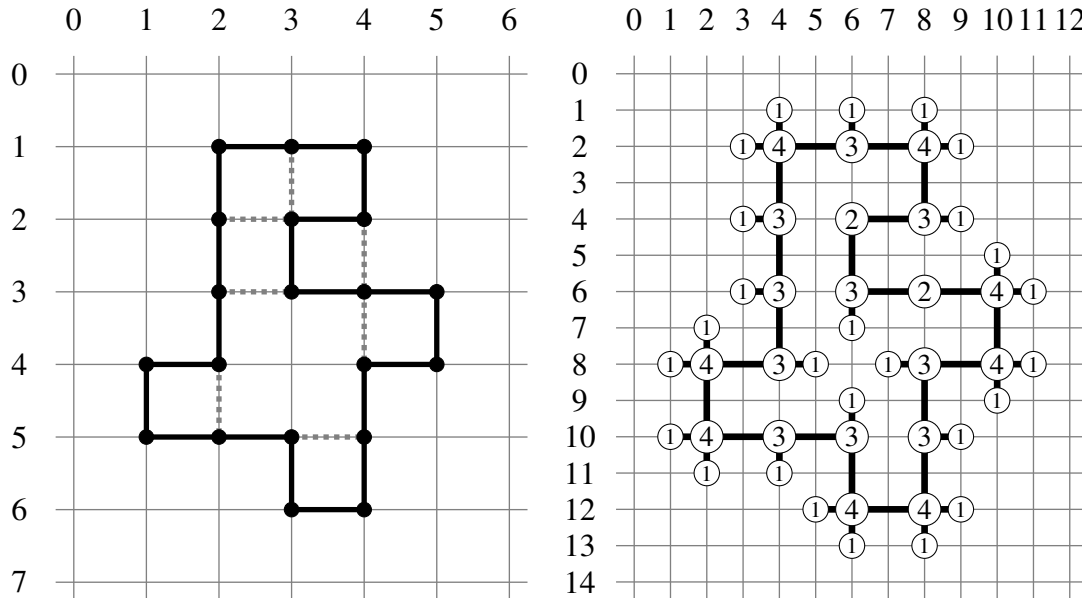


Figure 5: Reduction from the problem of Hamiltonian circuits in grid graphs (left) to the problem of Hashiwokakero (right).

It remains to be proven that any puzzle constructed in this way has a solution if and only if the corresponding grid graph has a Hamiltonian circuit. To prove this, we will examine all possible types of big islands, which are those with 4, 3, 2, 1, or 0 neighbouring big islands.

In the following, the term *neighbouring big islands* refers to the big islands at a distance of two and *neighbouring small islands* to the small islands at a distance of one for some big island.

We begin with those big islands that have 4, 3, or 2 neighbouring big islands. By construction, a big island with 4 neighbouring big islands always has a bridge constraint of 2. Hence, two bridges must go from this island to neighbouring big islands.

A big island with 3 neighbouring big islands has 1 neighbouring small island and a bridge constraint of 3. To satisfy the connectivity rule of Hashiwokakero, one bridge must be connected to the small island. This is necessary, as each small island can only be connected to its corresponding big island. It cannot be connected to another big island, as there is always a small island in between. It can also not be connected to another small island, as such a pair would remain disconnected from the rest of the puzzle. So, after connecting the big island to the small island, two bridges remain for the big island to go to the neighbouring big islands.

Similarly, a big island with 2 neighbouring big islands has 2 neighbouring small islands and a bridge constraint of 4. To ensure full connectivity, two bridges must connect to the small islands, leaving two bridges to go to neighbouring big islands.

In all three cases, exactly two bridges remain to connect to neighbouring big islands. These two bridges must go to two distinct islands. A double bridge would prevent full connectivity since neither island would have any remaining bridges to connect to the others. Therefore, only single bridges are possible, and each big island must be connected to two other big islands.

We now turn big islands with 1 and then 0 neighbouring big islands. A big island with 1 neighbouring big island has 3 neighbouring small islands and a bridge constraint of 5. To ensure full connectivity, three bridges go to the small islands, leaving two bridges. Since this type of island has only one neighbouring big island, the two bridges must both connect to that neighbour, forming a double bridge. However, as we just found, any possible neighbour has only two bridges to connect to big islands. Thus, using a double bridge would disconnect this pair from the rest of the puzzle, making the Hashiwokakero instance unsolvable. This is consistent with the Hamiltonian circuit problem, since a point with only one point at unit distance cannot be part of a circuit in a unit-distance graph.

Finally, if a big island has 0 neighbouring big islands, it has 4 neighbouring small islands and a bridge constraint of 6. After connecting this island to all four small islands, two bridges remain that cannot be connected to any other island, rendering the Hashiwokakero instance unsolvable. This is again consistent with the Hamiltonian circuit problem, as a point with no neighbours at unit distance cannot be part of a circuit in a unit-distance graph.

Considering all cases shows that a valid solution exists only if each big island has 2, 3, or 4 neighbouring big islands, and thus two remaining bridges, as well as two distinct neighbours for these bridges to connect to. Now, if the puzzle has a valid solution, then



two things are implied. First, the bridge constraint of each island is fulfilled, which means that each big island is connected to other big islands with exactly two bridges. And second, connectivity is fulfilled, which means that these two other big islands are distinct. So each big island, representing a point in the grid graph, is connected to exactly two distinct other big islands that also represent points in the grid graph. Now, the resulting bridges between big islands correspond precisely to the edges of a Hamiltonian circuit in the original grid graph. Following these connections and recording the corresponding points as vertices and the bridges as edges, we can construct a Hamiltonian circuit.

Conversely, if the Hashiwokakero instance is not solvable, then at least one big island cannot satisfy its bridge constraint or connectivity requirement, while adhering to the other puzzle rules. In this case, the corresponding point in the grid graph cannot be part of a Hamiltonian circuit, as it lacks two points at a unit distance that are not already included in the circuit. Since a Hamiltonian circuit must contain every vertex in a graph, no such circuit can exist in the grid graph. This completes the proof of Lemma 3.2.  $\square$

With Lemma 3.2, it is established that the problem of Hamiltonian circuits in grid graphs can be reduced to the problem of Hashiwokakero in polynomial time. Since the problem of Hamiltonian circuits in grid graphs is NP-complete [15], this reduction implies that any problem in NP can be reduced to Hashiwokakero in polynomial time. By Definition 3.4, we can therefore immediately conclude the following theorem.

**Theorem 3.2 (NP-Hardness of Hashiwokakero)**

Hashiwokakero is NP-hard.

In the last two sections, we have established that Hashiwokakero is in NP and that it is also NP-hard, so by Definition 3.5, we have proven the following theorem.

**Theorem 3.3 (NP-Completeness of Hashiwokakero)**

Hashiwokakero is NP-complete.

The NP-completeness of Hashiwokakero has theoretical and practical implications. Due to its NP-completeness, any puzzle in NP can be reduced to Hashiwokakero in polynomial time. This means that, unless  $P=NP$ , there is no deterministic polynomial algorithm that is able to solve all Hashiwokakero instances in polynomial time. However, there may still be efficient, practical solvers. Since any NP-complete problem can be reduced to SAT in polynomial time [17], we can use SAT and SMT solvers. These solvers employ highly optimized algorithms and heuristics to handle many difficult instances effectively.

The following chapters examine these reductions from Hashiwokakero to SAT and SMT. In addition, we develop a dedicated solver based on heuristics that are specifically designed for the structure of the puzzle. Finally, we compare the running times of these three approaches when solving the NP-complete puzzle of Hashiwokakero.

## 4 Solving Hashiwokakero with SAT

This chapter presents the first approach to solving Hashiwokakero. The puzzle is reduced to a SAT problem and then solved using a SAT solver. First, we introduce the SAT problem by establishing the necessary terminology from propositional logic. This includes Boolean formulas, variable assignments, the semantic evaluation of formulas, satisfiability and finally the satisfiability problem. To use SAT solvers, formulas must often be formulated in conjunctive normal form (CNF). We thus define CNF and present the Tseitin encoding, which transforms any Boolean formula into CNF without exponential blow-up. Having defined the SAT problem and explained how any Boolean formula can be transformed into CNF, we then finally apply this theoretical foundation to encode and solve Hashiwokakero as a SAT problem.

### 4.1 SAT Problem

The satisfiability problem (SAT problem) is the problem of deciding whether a given formula is *satisfiable*. To define satisfiability, we first need to introduce some terminology. We begin with Boolean formulas, the language in which the SAT problem is formulated. The presentation of the following definitions 4.1-4.4 is adapted from Westreicher [18].

#### Definition 4.1 (Boolean Formula)

A Boolean formula is defined inductively along the following rules:

- The constants  $\top$  and  $\perp$  are Boolean formulas.  $\top$  represents true and  $\perp$  represents false.
- The variables  $x, y, \dots$  are Boolean formulas. They represent truth values ( $\top$  or  $\perp$ ).
- If  $\phi$  and  $\psi$  are Boolean formulas, then the following are also Boolean formulas:
  - Negation:  $\neg\phi$  (“not  $\phi$ ”)
  - Conjunction:  $\phi \wedge \psi$  (“ $\phi$  and  $\psi$ ”)
  - Disjunction:  $\phi \vee \psi$  (“ $\phi$  or  $\psi$ ”)
  - Implication:  $\phi \Rightarrow \psi$  (“ $\phi$  implies  $\psi$ ”)
  - Equivalence:  $\phi \Leftrightarrow \psi$  (“ $\phi$  is equivalent to  $\psi$ ”)
- Parentheses ‘(’ and ‘)’ may be used to clarify or change the precedence of operators.

We have now defined Boolean formulas, including variables as their basic building blocks. To give these variables meaning, we assign a truth value to each variable.

#### Definition 4.2 (Assignment)

Let  $\phi$  be a Boolean formula with variables  $Vars$ . An assignment for  $\phi$  is a function  $f : Vars \rightarrow \{\top, \perp\}$ , that assigns a truth value to each variable.

Having defined the syntax of Boolean formulas and the notion of variable assignment, we can now introduce the semantics of Boolean formulas. These are given by an evaluation function that maps a formula and a variable assignment to a truth value by recursively evaluating the subformulas.

**Definition 4.3 (Evaluation)**

Let  $\phi$  be a Boolean formula and  $f$  an assignment of truth values to the variables in  $\phi$ . Then the function  $eval$  is recursively defined by the following rules:

- $eval(\top) = \top$  and  $eval(\perp) = \perp$
- $eval(x) = f(x)$  for a variable  $x$
- $eval(\neg\phi) = \top$  iff  $eval(\phi) = \perp$
- $eval(\phi \wedge \psi) = \top$  iff  $eval(\phi) = \top$  and  $eval(\psi) = \top$
- $eval(\phi \vee \psi) = \top$  iff  $eval(\phi) = \top$  or  $eval(\psi) = \top$  (inclusive or)
- $eval(\phi \Rightarrow \psi) = \top$  iff  $eval(\phi) = \perp$  or  $eval(\psi) = \top$  (inclusive or)
- $eval(\phi \Leftrightarrow \psi) = \top$  iff  $eval(\phi) = eval(\psi)$

We have now defined the syntax and semantics of Boolean formulas. Based on this, we can formally introduce satisfiability.

**Definition 4.4 (Satisfiability)**

A propositional formula  $\phi$  is satisfiable if there exists an assignment  $f$  such that  $eval(\phi) = \top$  under  $f$ .

Having introduced satisfiability, we now define the satisfiability problem.

**Definition 4.5 (SAT Problem)**

Given a Boolean formula  $\phi$ . Is there any assignment that satisfies  $\phi$ ?

**Example 4.1**

The formula  $(x_1 \vee \neg x_2) \wedge \neg x_1$  is satisfiable. We can assign  $x_1 = \perp$  and  $x_2 = \perp$ . Then we have  $(\perp \vee \top) \wedge \top$  and thus  $\top \wedge \top$ , which evaluates to  $\top$ .

The SAT problem is NP-complete [17]. This implies that any problem in NP can be reduced to SAT in polynomial time. Consequently, many real-world problems, such as verification in hardware and software, planning, scheduling, and combinatorial design, can be formulated as SAT instances [19]. This makes SAT a central problem in computer science, for which efficient solution methods are of great practical importance.

Despite the theoretical complexity, SAT instances can often be solved efficiently using SAT solvers. These algorithms are specialised for solving SAT problems. They combine sophisticated heuristics and optimisation techniques to handle even large and complex instances effectively. Many SAT solvers require formulas to be in conjunctive normal

form (CNF). To define CNF, we first introduce its building blocks. We begin with literals, the smallest entities in CNF. The following definitions 4.6-4.8 are adapted from Biere et al. [19].

**Definition 4.6 (Literal)**

A literal  $l$  is either a Boolean variable  $v$  or its negation  $\neg v$ .

**Example 4.2**

The formula  $(x_1 \vee \neg x_2) \wedge \neg x_1$  contains the literals  $x_1$ ,  $\neg x_2$  and  $\neg x_1$ .

These literals can then be combined into clauses, where at least one literal must be true for the clause to evaluate as true.

**Definition 4.7 (Clause)**

A clause  $c$  is a disjunction of literals  $\bigvee_i l_i$ .

A clause of length zero is called an empty clause. It is unsatisfiable, as it evaluates to false under any assignment.

**Example 4.3**

The formula  $(x_2 \vee x_4 \vee \neg x_3)$  is a disjunction of the literals  $x_2$ ,  $x_4$  and  $\neg x_3$  and is thus a clause.

These clauses are now combined into formulas in conjunctive normal form. These formulas evaluate as true if every single clause evaluates as true.

**Definition 4.8 (Conjunctive Normal Form (CNF))**

A formula  $f$  in conjunctive normal form (CNF) is a conjunction of clauses  $\bigwedge_j c_j$ .

**Example 4.4**

The formula  $(x_1 \vee \neg x_2) \wedge (\neg x_1) \wedge (x_2 \vee x_4 \vee \neg x_3)$  is a conjunction of the clauses  $x_1 \vee \neg x_2$ ,  $\neg x_1$  and  $x_2 \vee x_4 \vee \neg x_3$ , and is thus in conjunctive normal form.

Any Boolean formula can be transformed into an equisatisfiable formula in CNF [19]. As a direct translation using Boolean algebra generally leads to exponentially large formulas [19], we use the Tseitin encoding. The Tseitin encoding introduces a linear number of fresh variables to produce a CNF formula of linear size. It works by replacing subformulas with new variables and enforcing their equisatisfiability by introducing additional clauses [19]. We illustrate this approach with an example.

**Example 4.5**

For the formula  $a \Rightarrow (b \wedge c)$ , we want to find an equivalent formula in conjunctive normal form. We first introduce a new variable  $x_1$  for  $(b \wedge c)$ . We now have to add the equivalence  $x_1 \Leftrightarrow (b \wedge c)$ . This equivalence can be turned into two implications  $(b \wedge c) \Rightarrow x_1$  and  $x_1 \Rightarrow (b \wedge c)$ . And both of these can then be written as disjunctions  $\neg(b \wedge c) \vee x_1$  and  $\neg x_1 \vee (b \wedge c)$ . Now we can use De Morgan's Laws to obtain  $(\neg b \vee \neg c \vee x_1)$  from the first formula and the distributive law to get  $(\neg x_1 \vee b)$  and  $(\neg x_1 \vee c)$  from the second formula.

All of these are clauses. This leaves us with the formula  $a \Rightarrow x_1$ . We introduce a new variable  $x_2$  with  $x_2 \Leftrightarrow (a \Rightarrow x_1)$ . This equivalence can be rewritten to  $x_2 \Rightarrow (a \Rightarrow x_1)$  and  $(a \Rightarrow x_1) \Rightarrow x_2$ . We now rewrite these as disjunctions to obtain  $\neg x_2 \vee (a \Rightarrow x_1)$  and  $\neg(a \Rightarrow x_1) \vee x_2$ . For the first formula, we rewrite the implication as a disjunction to get  $(\neg x_2 \vee \neg a \vee x_1)$ . The second formula needs a few more steps. We first rewrite the implication as a disjunction to get  $\neg(\neg a \vee x_1) \vee x_2$ , then we use De Morgan's Laws to get  $(a \wedge \neg x_1) \vee x_2$ , and finally, with distributivity, we get the clauses  $(a \vee x_2)$  and  $(\neg x_1 \vee x_2)$ . Now we formulate all these clauses as a conjunction to get  $(\neg b \vee \neg c \vee x_1) \wedge (\neg x_1 \vee b) \wedge (\neg x_1 \vee c) \wedge (\neg x_2 \vee \neg a \vee x_1) \wedge (a \vee x_2) \wedge (\neg x_1 \vee x_2)$ . This formula is now in CNF and equisatisfiable to the initial formula  $a \Rightarrow (b \wedge c)$ .

With the Tseitin encoding, we have introduced a method for transforming arbitrary Boolean formulas to CNF, the input format for many SAT solvers. This completes the necessary theoretical background.

In the following section, we reduce Hashiwokakero to a SAT problem by expressing the puzzle constraints as Boolean formulas and transforming them into CNF using the Tseitin encoding. The resulting SAT problem is then efficiently solved using a SAT solver.

## 4.2 Hashiwokakero as a SAT Problem

We will now reduce Hashiwokakero to a SAT problem using the notation introduced in Section 2.1. This is done by first introducing Boolean variables, then encoding all Hashiwokakero rules as CNF constraints, and finally solving the resulting CNF formula with a SAT solver. The resulting variable assignment can then be translated back into a solution for Hashiwokakero. We begin by defining Boolean variables that correspond to the bridges in the puzzle.

To generate an efficient SAT encoding, variables are only introduced for bridges between neighbouring islands, as defined in Section 2.1. This is reasonable, as any solution candidate with bridges that do not connect two neighbouring islands violates some Hashiwokakero rule and thus does not need to be considered. Each bridge variable is named after the bridge it represents.

### Definition 4.9 (Bridge Variables)

For each pair of neighbouring islands  $a, c \in I$ , we introduce the Boolean variables  $B_{ac}^1$  and  $B_{ac}^2$ . They represent the first and second bridge between these islands. The variables are interpreted as follows:

$$B_{ac}^i = \begin{cases} \text{True,} & \text{if the bridge } B_{ac}^i \text{ is set in a solution to Hashiwokakero,} \\ \text{False,} & \text{otherwise.} \end{cases}$$

$B_{ac}^i$  and  $B_{ca}^i$  both denote the same variable.

Based on these variables, we now encode constraints to ensure that any satisfying assignment also represents a valid solution to Hashiwokakero.

Before implementing the constraints for the Hashiwokakero rules, we first add the preliminary constraint that a second bridge can only be *set* if the corresponding first bridge is also *set*.

#### Constraint 4.1

For each pair of bridge variables between islands  $a, c \in I$ , we add the constraint:

$$B_{ac}^1 \vee \neg B_{ac}^2$$

This constraint ensures that a second bridge is only *set* when the corresponding first bridge is also *set*. Consequently, two islands may have no bridge, only the first bridge, or both bridges *set*, but never the second bridge without the first. Because of this constraint, it is sufficient to only consider the first bridges in the constraints related to collisions and connectivity. This reduces the total number of constraints required.

With this preliminary constraint in place, we now proceed to the full encoding. We consider each rule of Hashiwokakero separately and translate it into SAT constraints.

#### 4.2.1 Rules 1 to 5

**Rule 1: Bridges must begin and end at distinct islands** This rule can be enforced in several ways. The simplest approach, which we adopt here, is to introduce variables only for bridges between two distinct islands. Alternatively, bridge variables could be defined at all grid intersections together with additional constraints to ensure that bridges only start and end at distinct intersections with islands. However, this would increase the number of variables and complicate the encoding, which is why we have chosen to define bridge variables only for bridges starting and ending at islands. A satisfying assignment to the SAT problem thus already satisfies Rule 1 by the definition of the bridge variables.

**Rule 2: Brides may only run horizontally or vertically** A similar consideration applies to Rule 2. While it would be possible to introduce additional variables, allowing the SAT solver to decide whether a bridge is horizontal or vertical, this would significantly increase the complexity of the encoding. As SAT solvers only operate on Boolean variables, enforcing this rule would require numerous additional variables to represent row and column positions, as well as many additional constraints. For this reason, we have chosen to define variables only for horizontal and vertical bridges. A satisfying assignment to the SAT problem thus already satisfies Rule 2 by definition of the bridge variables.

**Rule 3: Bridges must not cross any other bridges or islands** To enforce Rule 3, we must ensure that bridges do not cross other bridges or islands. By definition of the bridge

variables, it is already ensured that bridges do not cross any islands, as bridge variables are only defined between neighbours. But the problem of bridges crossing other bridges still needs to be handled.

To avoid collisions between bridges, we only need to consider the first bridge in each pair. As a second bridge can only be *set* when a first bridge is already *set* by Constraint 4.1, this is sufficient to avoid collisions.

We identify collisions in two steps: First, we trace every vertical bridge and record its name at each intersection it passes. Second, we trace all horizontal bridges. If a horizontal bridge passes through an intersection already marked with a vertical bridge, we record the bridge variables corresponding to these bridges.

#### Constraint 4.2

For all colliding bridges  $B_{ac}^1, B_{de}^1$ , we add the constraint:

$$\neg B_{ac}^1 \vee \neg B_{de}^1$$

This ensures that both bridges cannot be *set* simultaneously, thus preventing the collision.

**Rule 4: At most two bridges may connect any pair of islands** Rule 4 is trivially fulfilled, as there are only two bridge variables between each pair of neighbouring islands, one for the first and one for the second bridge. That way, we can only have 0, 1, or 2 bridges *set* between that pair of islands. We thus do not need any additional constraints for Rule 4.

**Rule 5: The number of bridges connected to each island must match the number indicated on that island** For an island  $a$ ,  $num(a)$  denotes the bridge constraint. Depending on this bridge constraint, some combinations of bridges are allowed, while other combinations are not. To only allow combinations that satisfy the bridge constraint, we define some functions that enforce one of the combinations where the bridges are correct. To do this, we first need to define for each island a set of all bridge variables that correspond to bridges connected to it.

#### Definition 4.10 (Bridge Sets)

Let  $a \in I$  be an island. The set  $bridges_a$  is defined as follows:

$$bridges_a = \{B_{am}^i \mid \text{for some island } m \in neighbours(a), i \in \{1, 2\}\}$$

Now we can define a family of combinatorial functions that select all  $k$ -fold combinations of elements from a set.

#### Definition 4.11 (Exactly K)

For  $A = \{a_1, \dots, a_n\}$ , the function  $Exactly_k(A)$  returns a formula that evaluates to *True* if exactly  $k$  variables evaluate to *True*.

$$Exactly_k(A) = \bigvee_{\substack{S \subseteq \{1, \dots, n\} \\ |S|=k}} \bigwedge_{i \in S} a_i \wedge \bigwedge_{j \in \{1, \dots, n\} \setminus S} \neg a_j$$

Now we can add a constraint for each island, to ensure that all bridge constraints are fulfilled.

### Constraint 4.3

For each island  $a \in I$  with bridges  $bridges_a$  and bridge constraint  $num(a)$ , we add the constraint:

$$Exactly_{num(a)}(bridges_a)$$

We use the Tseitin encoding to transform these constraints into CNF. In our implementation, we encode this cardinality constraint using the method `CardEnc.equals` from the module `pysat.card.CardEnc` of the PySAT toolkit [20].

Having encoded Rules 1 to 5, we now try to find an efficient encoding for Rule 6.

#### 4.2.2 Rule 6: Connectivity

Rule 6 states that all islands must be connected so that each island can be reached from any other island. This rule is the most challenging to encode as a SAT constraint. Since the choice of encoding might impact the solver performance, we consider three different approaches.

**Connectivity by enumerating connected puzzles** The first approach is to list all possible connected configurations and ensure that the solution is among them. This is achieved through a recursive search that starts from a single island and explores all combinations of outgoing bridges. For each combination, the newly reached islands are listed, and the process is repeated from these islands. This continues until all possible configurations have been explored. For each connected configuration, the assignment of variables is stored. This procedure yields a complete list of assignments that lead to connected configurations, represented as a disjunction of conjunctions. This list must then be transformed into CNF. However, this approach has factorial complexity and is therefore not feasible for larger puzzles. A more efficient encoding of connectivity is required.

**Connectivity by reachability** The idea of this approach is to ensure global connectivity via local constraints. Instead of enumerating all connected configurations, we enforce that every island is connected to another island that is closer to a designated starting island. Distance is measured by the number of bridges along the path: The starting island has distance 0, islands directly connected to it have distance 1, and so on. With each bridge along the paths the distances increase by one. By enforcing that every island is connected to an island with a strictly smaller distance, we ensure reachability from the starting island via a chain of bridges. This reduces the problem of connectivity to local constraints between



neighbouring islands.

To enforce this reachability constraint, we introduce distance variables, assigning a number to each island. This number represents the distance of that island from a designated starting island  $s \in I$ , measured as the number of bridges in a path to the starting island. These numbers range from 0 to  $n - 1$ , where  $n$  is the number of islands in the puzzle, since  $n - 1$  is the maximum possible distance from any island to the starting island. If an island has multiple paths to the starting island, any valid distance value is acceptable. As SAT solvers only operate on Boolean variables, we have to encode the distance variables accordingly. Thus, for each island, we create a distance variable for each possible distance.

**Definition 4.12 (Distance variables)**

For each island  $a \in I$ , we introduce the distance variables  $N_{a0}, N_{a1}, \dots, N_{a(n-1)}$ . These variables are defined as follows:

$$N_{ai} = \begin{cases} \text{True,} & \text{if } a \text{ has a distance of } i \text{ to the starting island } s \in I, \\ \text{False,} & \text{otherwise.} \end{cases}$$

Since each island must have exactly one distance to the starting island, exactly one of the corresponding distance variables must be true, while all others must be false. To enforce this constraint, we use the combinatorial function  $Exactly_1$  as defined in Definition 4.11.

**Constraint 4.4**

For each island  $a \in I$ , the following constraint is added:

$$Exactly_1(\{N_{ai} \mid \text{for } i \in \{1, \dots, n-1\}\})$$

It is translated into CNF using the Tseitin encoding. In our implementation, we encode this cardinality constraint using the method `CardEnc.equals` from the module `pysat.card.CardEnc` of the PySAT toolkit [20].

Having ensured that each island is assigned exactly one distance value, we select an arbitrary island  $s \in I$  to serve as the starting island. This island is assigned a distance of 0, while the other islands must have greater distances, so that each island needs to be connected to  $s$  via a path of bridges. To achieve this, we add the following constraints.

**Constraint 4.5**

For the starting island  $s$ , we add the clauses:

$$\begin{aligned} &N_{s0}, \\ &\neg N_{si} \text{ for } i \in \{1, \dots, n-1\} \end{aligned}$$

For each island  $a \in I$  with  $a \neq s$ , we add the clause:

$$\neg N_{a0}$$

Based on these constraints, we can now enforce local connectivity. Each island, except the starting island, must be connected via a bridge to at least one neighbouring island, whose distance to the starting island is exactly one less. This ensures that distances decrease step by step along a path towards the starting island. The corresponding constraint is given below.

**Constraint 4.6**

For each island  $a \in I$ ,  $a \neq s$  with neighbours  $b, c, d, e \in I$  and for each number  $x \in \{1, \dots, n-1\}$ , we add the constraint:

$$N_{a(x)} \Rightarrow (N_{b(x-1)} \wedge B_{ab}^1) \vee (N_{c(x-1)} \wedge B_{ac}^1) \vee (N_{d(x-1)} \wedge B_{ad}^1) \vee (N_{e(x-1)} \wedge B_{ae}^1)$$

It is translated into CNF using the Tseitin encoding.

Since Constraint 4.4 enforces that every island must be assigned a distance value, Constraint 4.6 implies that each island is connected to at least one island whose distance value is exactly one less than its own. And since the starting island is the only island with a distance value of 0 by Constraint 4.5, it follows that every island must be connected to the starting island.

After encoding all constraints, the SAT solver is used to find a variable assignment satisfying these constraints. Any satisfying assignment corresponds to a valid solution of the Hashiwokakero instance.

**Connectivity by a mixed approach** Finally, we present an alternative approach for Rule 6 that combines SAT solving with a simple verification function. This approach is motivated by the observation that Rule 6 is significantly more complex to encode than the other rules. To improve performance, we temporarily omit Rule 6 and use the SAT solver to find an assignment that satisfies the remaining constraints. We then use a separate verification function to check whether the resulting candidate solution satisfies the connectivity requirement. If connectivity is not satisfied, the candidate solution is excluded, and the process is repeated until a valid solution is found. This approach is illustrated in Figure 6.

For this approach, we do not encode any additional constraints for Rule 6. Instead, we first use the SAT solver to find an assignment that satisfies Rules 1-5. Only then is the connectivity required by Rule 6 checked for this assignment, using a breadth-first search algorithm.

The algorithm starts from a designated starting island. From this island, all bridges, whose corresponding bridge variables have been assigned the value true, are traced to reach

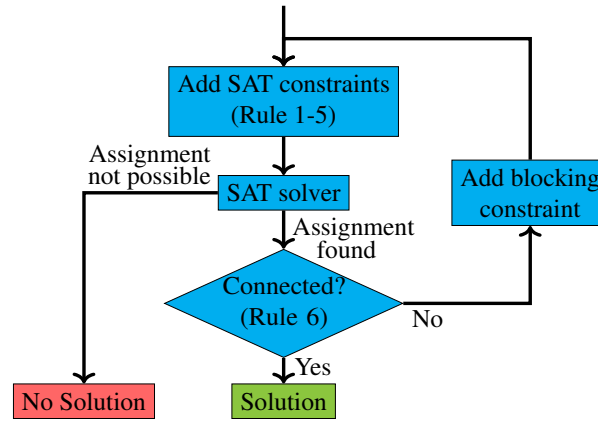


Figure 6: Overview of the mixed approach that combines SAT solving with a dedicated verification function for Rule 6.

adjacent islands. These islands are added to a list of visited islands, which initially contains only the starting island. All previously unvisited islands are additionally added to a queue to become the new starting islands in the following steps. This process is repeated until the queue is empty, at which point all islands connected to the initial starting island have been visited.

The puzzle is fully connected if the number of visited islands equals the total number of islands in the puzzle. In this case, the assignment represents a valid solution to the puzzle. Otherwise, the assignment is excluded from further SAT solver runs by adding a *blocking constraint*. This constraint is created by negating all the bridge variables in the current assignment and combining them into a disjunction. This ensures that at least one variable differs from the current assignment, thus preventing the SAT solver from returning the same invalid assignment again.

The solver is then re-executed, and this process is repeated until either a valid solution is found or the SAT solver fails to find any other assignment that satisfies Rules 1-5.

A similar strategy can also be used to find multiple solutions. It can be applied to each of the three approaches described above. Once a solution has been found, it is stored, and a blocking constraint is added to prevent the SAT solver from returning the same assignment again. The solver is thus forced to search for a different satisfying assignment, if one exists. For the mixed approach, the connectivity check for Rule 6 must be performed for each candidate solution. This process is repeated until the solver cannot find another assignment that satisfies the constraints. All assignments found in this way correspond to valid solutions of the Hashiwokakero instance. This strategy identifies all valid solutions to the puzzle instance.

Using any of the approaches above, we have now found at least one solution to the SAT problem. Translating this SAT solution to a Hashiwokakero solution is straightforward. The SAT solver returns an assignment for all Boolean variables. The bridges whose corresponding variables have been assigned true are exactly those bridges that form the Hashiwokakero solution.

We have now presented three approaches to solving Hashiwokakero using a SAT solver. The performance of these approaches is evaluated in Chapter 7. However, encoding the problem solely with Boolean variables has been cumbersome. In the next chapter, we therefore introduce an alternative approach that allows the problem to be formulated more directly and naturally.

## 5 Solving Hashiwokakero with SMT

As discussed in the previous chapter, SAT solvers are highly efficient for problems expressed in Boolean formulas. However, many real-world problems are difficult to encode in this way, as higher-level concepts, such as integers or arithmetic operations, must be represented using large sets of Boolean variables and constraints. This increases both the size and complexity of the encoding, making it less intuitive. This issue is particularly evident in the SAT encoding of Rule 6, for which a large number of clauses are required to model integers and arithmetic operations. In contrast, satisfiability modulo theories (SMT) supports such concepts directly through built-in theories, enabling more natural and compact encodings.

### 5.1 SMT Problem

The SMT problem extends the Boolean satisfiability problem by incorporating first-order theories, such as equality reasoning, arithmetic, fixed-size bit vectors, arrays, and quantifiers [21]. That way, complex concepts can be handled directly, without translation into Boolean formulas. This leads to more abstract and high-level encodings compared to SAT. The formal definition of the SMT problem is as follows:

**Definition 5.1 (SMT Problem [22])**

Given a first-order formula. Is this formula satisfiable with respect to some logical theory?

In this sense, SMT extends SAT. While both problems address the question of satisfiability, SMT operates on a richer logical domain, thereby enabling more expressive modelling.

SMT is of major practical importance due to its numerous applications in software and hardware verification, including extended static checking, predicate abstraction, test case generation, and bounded model checking over infinite domains [21]. To solve these problems efficiently, SMT solvers combine SAT-solving techniques with specialised theory solvers, thereby extending the capabilities of classical SAT solvers to richer logic domains. There are two main approaches to combining SAT solving and theory reasoning: an eager approach and a lazy approach. We follow the explanation given by Biere et al. [19].

In the eager approach, the formula is translated into an equisatisfiable Boolean formula using theory-specific encodings. This translation can significantly increase the size of the formula, but it can then be given to any off-the-shelf SAT solver.

In contrast, the lazy approach avoids a complete upfront translation. Instead, it integrates SAT solving with decision procedures for the underlying theories, allowing the components to interact during the solving process.

As SMT solvers support theories such as integer arithmetic, modelling Hashiwokakero is significantly easier in SMT than in SAT. We will now proceed with this reduction.

## 5.2 Hashiwokakero as an SMT Problem

We now reduce Hashiwokakero to an SMT problem using the notation introduced in Section 2.1. Similar to the SAT reduction, we first introduce bridge variables, after which we encode the Hashiwokakero rules as constraints, and finally, we use an SMT solver to find an assignment that satisfies these constraints. This assignment is then translated back into a valid Hashiwokakero solution.

We begin with the definition of bridge variables. Similar to the SAT approach, we only introduce variables for bridges between neighbouring islands. While in SAT, two Boolean variables represent the two bridges between some islands, in the SMT approach, we use a single integer variable for each pair of bridges. This variable encodes the number of bridges that are *set* between the islands.

### Definition 5.2 (Bridge variables)

For each pair of neighbouring islands  $a, c \in I$ , we introduce the bridge variable  $B_{ac}$ . This variable is interpreted as follows:

$$B_{ac} = x, \text{ where } x \text{ is the number of bridges } \textit{set} \text{ between } a \text{ and } c$$

$B_{ac}$  and  $B_{ca}$  both denote the same variable.

Based on these numeric bridge variables, we now encode constraints to ensure that any assignment that satisfies all constraints represents a valid solution to Hashiwokakero.

### 5.2.1 Rules 1 to 5

**Rules 1 and 2: Bridges must begin and end at distinct islands, and they may only run horizontally or vertically** Similar to the SAT approach, Rules 1 and 2 are already fulfilled by the definition of the bridge variables. As bridge variables are only introduced between neighbouring islands, it is first ensured that bridges start and end at distinct islands and, second, that bridges run only horizontally or vertically.

**Rule 3: Bridges must not cross any other bridges or islands** To enforce Rule 3, we must ensure that bridges do not cross other bridges or islands. Since bridge variables are only introduced between neighbouring islands, bridges cannot cross islands by definition. It remains to prevent collisions between bridges.

To identify colliding bridges, we use the same approach as in Chapter 4. For each pair of colliding bridges, we then add a constraint to ensure that both bridges cannot be *set* at the same time. Using the SMT bridge variables, this is expressed by enforcing that, whenever one bridge variable is greater than zero, the other must be equal to zero.

### Constraint 5.1

For all colliding bridges  $B_{ac}, B_{de}$ , we add the constraint:

$$(B_{ac} > 0 \Rightarrow B_{de} = 0) \text{ and } (B_{de} > 0 \Rightarrow B_{ac} = 0)$$

**Rule 4: At most two bridges may connect any pair of islands** Rule 4 requires that each pair of islands is connected by at most two bridges. We restrict the bridge variables accordingly, by adding a constraint to ensure that each bridge variable is either 0, 1, or 2.

### Constraint 5.2

For each pair of neighbouring islands  $a, c \in I$ , we add the constraint:

$$B_{ac} = 0 \vee B_{ac} = 1 \vee B_{ac} = 2$$

**Rule 5: The number of bridges connected to each island must match the number indicated on that island** For Rule 5, we must ensure that the number of *set* bridges connected to each island  $a \in I$  matches its bridge constraint  $num(a)$ . Similar to the SAT approach, we define a set of connected bridges for each island.

### Definition 5.3 (Bridge Sets)

Let  $a \in I$  be an island. The set  $bridges_a$  is defined as follows:

$$bridges_a = \{B_{am} \mid \text{for some island } m \in neighbours(a)\}$$

Since the bridge variables directly represent the number of bridges between two islands, the total number of bridges connected to an island  $a \in I$  is the sum of the bridge variables in  $bridges_a$ . This is encoded in the following constraint.

### Constraint 5.3

For each island  $a \in I$ , we add the constraint:

$$SUM(bridges_a) = num(a)$$

With this, Rules 1 to 5 are fully encoded as SMT constraints. We now turn to Rule 6 using the ideas from the SAT approach.

## 5.2.2 Rule 6: Connectivity

Rule 6 states that all islands must be connected so that each island can be reached from any other island. Since the choice of encoding for Rule 6 affects the running time, we consider two different approaches: Connectivity by reachability and connectivity via a mixed approach. Both follow the same idea as their counterparts in SAT formulation.

**Connectivity by reachability** For connectivity by reachability, we use the same idea as in the SAT approach. We enforce that every island is connected to another island that is

closer to a designated starting island. As before, distance is measured by the number of bridges along the path to the starting island, which is the only island with a distance of 0. To model these distances, we introduce a distance variable for each island. Unlike SAT solvers, SMT solvers support integers, so the distances can be represented with a single number variable, instead of multiple Boolean variables.

**Definition 5.4 (Distance variables)**

For each island  $a \in I$ , we introduce the distance variable  $N_a$ . It is interpreted as follows:

$$N_a = x, \text{ where } x \text{ is the distance of } a \text{ to the starting island } s \in I$$

We then select an arbitrary island as the starting island  $s \in I$ , and assign it a distance of 0.

**Constraint 5.4**

For the starting island  $s \in I$ , we add the constraint:

$$N_s = 0$$

Next, every island except the starting island must be connected to a neighbouring island whose distance variable is smaller by one.

**Constraint 5.5**

For each island  $a \in I$  with neighbouring islands  $b, c, d, e \in I$ , we add the following constraint:

$$\begin{aligned} &((N_a = N_b + 1) \wedge (B_{ab} > 0)) \vee ((N_a = N_c + 1) \wedge (B_{ac} > 0)) \\ &\vee ((N_a = N_d + 1) \wedge (B_{ad} > 0)) \vee ((N_a = N_e + 1) \wedge (B_{ae} > 0)) \end{aligned}$$

Since the starting island is the only island with a distance of 0, every other island must be connected to it by a path of bridges through islands with decreasing distance variables. This guarantees that all islands in the puzzle are connected.

This completes the reachability encoding of Hashiwokakero. An SMT solver is now used to find an assignment that satisfies all constraints. Any such assignment corresponds to a valid solution to the instance of Hashiwokakero.

**Connectivity by a mixed approach** The mixed approach also follows the same idea as its SAT counterpart. Only Rules 1 to 5 are encoded as SMT constraints, and an SMT solver is used to find an assignment satisfying them. The solution corresponding to this assignment is then checked for connectivity using the same breadth-first search algorithm as in the SAT approach. If the assignment represents a connected puzzle, it corresponds to a valid solution. Otherwise, a blocking constraint is added to ensure that at least one bridge variable differs in the subsequent solver runs. This approach is illustrated in Figure 7.

Finding multiple solutions is also done in the same way as in the SAT approach. Whenever a valid solution is found, it is stored, and a blocking constraint is added. This forces the



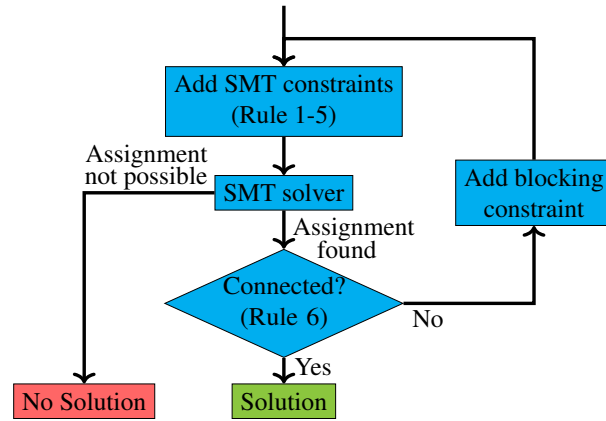


Figure 7: Overview of the mixed approach that combines SMT solving with a dedicated verification function for Rule 6.

solver to search for a different assignment in the next run. In the mixed approach, the connectivity check for Rule 6 must be performed for each assignment found by the solver. This process is repeated until no further satisfying assignment is found. All SMT solutions found during this process correspond to valid solutions of the Hashiwokakero instance. All valid solutions are identified in the process.

Translating an SMT solution back to Hashiwokakero is straightforward. A bridge variable with assigned value 1 represents a single bridge between the islands denoted in its subscript, while a bridge variable with assigned value 2 represents a double bridge. Together, these bridges form the solution to the instance of Hashiwokakero.

## 6 Solving Hashiwokakero with Deductions

The solver with deductive reasoning is after SAT solving and SMT solving, a third approach to solve Hashiwokakero puzzles. The idea behind the solver is to iteratively apply a set of deduction rules until no new deductions can be made, and then, if necessary, to interleave the iterations with a backtracking-based guessing strategy to reach a solution for unique puzzles and puzzles with multiple solutions. In this chapter, the structure of the algorithm will be explained, together with the deduction rules that are used to solve the puzzle.

### 6.1 Foundations

To get started, we first need terminology to speak about the islands and bridges in the puzzles. We build on the terminology that is already defined in Section 2.1. As introduced there, the bridge constraint of an island  $a \in I$  is denoted by  $num(a)$ . However, in the algorithm, we need to refer to the number of bridges that are still required to fulfil the bridge constraint, after some bridges have already been *set*. To differentiate this number of remaining bridges from the original bridge constraint, we call it the *decreased bridge constraint* or  $dec\_num(a)$ . When we work with bridges, we only *set* a second bridge when the first bridge has already been *set*, and we only *unset* a first bridge when the second bridge is also *unset*.

Having introduced the relevant terminology, we now define several sets on which the algorithm operates. For each island, we define *bridge sets* that contain all *undecided* bridges connected to this island. Initially, all bridges are *undecided* and thus included in the bridge sets of their two island endpoints. With each iteration of the algorithm, bridges are removed from these bridge sets whenever they are *set* or *unset*. Only bridges that are *undecided* remain in the bridge sets.

#### Definition 6.1 (Bridge Sets)

Let  $a \in I$  be an island with neighbouring islands  $N \subseteq I$ :

For each  $n \in N$ ,  $bridges_{an} = \{B_{an}^1, B_{an}^2\}$  contains all *undecided* bridges between  $a$  and  $n$ .

The set  $bridges_a = \bigcup_{n \in N} bridges_{an}$  contains all *undecided* bridges connected to  $a$ .

In addition to the bridge sets, we maintain for each bridge  $B_{ac}^i$  a set  $collisions_{B_{ac}^i}$  of all bridges  $B_{de}^j$  collides with.

We identify the collisions in the same way as in the previous chapters. Again, it is sufficient to consider the first bridges between islands. First, we trace every vertical bridge and record the name of the bridge at each intersection it passes. Second, we trace all horizontal bridges. Whenever a horizontal bridge  $B_{ac}^1$  passes through an intersection already occupied by a vertical bridge  $B_{de}^1$ , we have found a collision. For each bridge, all bridges it collides with are added to its *collision* set. So  $B_{de}^1$  and  $B_{de}^2$  are added to the sets  $collisions_{B_{ac}^1}$  and  $collisions_{B_{ac}^2}$  and vice versa.

**Definition 6.2 (Collision Sets)**

For any bridge  $B_{ac}^i$ ,  $i \in \{1, 2\}$ , the set  $collisions_{B_{ac}^i}$  contains all bridges that the bridge  $B_{ac}^i$  can collide with.

We now have bridge sets and collision sets that keep track of the local situation of the islands and bridges. But to use Hashiwokakero Rule 6 of connectivity, we also need a way to represent the global situation of the puzzle. For this, we keep track of *components*. *Components* are connected subgraphs, that are not part of a bigger subgraph. For each island, we track which component it belongs to. For each component, we store all islands that are part of this component, together with *open edges* and *open islands*. *Open edges* are those bridges that are currently *undecided*, but would connect this component to another component. *Open islands* are all islands in the component, that have at least one *open edge* in their bridge sets.

**Definition 6.3 (Component Sets)**

The set *components* contains all components in the puzzle. For each component  $c_j \in components$ , we define the following sets:

- $islands_{c_j}$ : Contains all islands in the component  $c_j$ .
- $open\_edges_{c_j}$ : Contains all *undecided* first bridges  $B_{ab}^1$  with  $a \in islands_{c_j}$ ,  $b \in islands_{c_k}$  and  $j \neq k$ .
- $open\_islands_{c_j}$ : Contains all islands  $a \in islands_{c_j}$ , s.t.  $B_{an}^1 \in open\_edges_{c_j}$  for some island  $n \in I$ .

For each island  $a \in I$ ,  $component(a)$  specifies the component  $c_j$  for which  $a \in islands_{c_j}$ .

When the algorithm starts, each island forms its own component, as no islands are connected in the beginning. With each deduction, we make changes to the state of the puzzle by *setting* and *unsetting* bridges.

Whenever some deduction leads us to *set* a bridge  $B_{ac}^i$ , we know that this bridge is present in the solution. We remove it from all bridge sets it is part of and decrease for both island endpoints  $a, c \in I$  the *decreased bridge constraints*  $dec\_num(a)$  and  $dec\_num(c)$  by one. Additionally, we *unset* all bridges in the set  $collisions_{B_{ac}^i}$ , as collisions are not allowed by Hashiwokakero Rule 3. Then, if the new bridge connects two previously distinct components of the puzzle, those components are merged. The islands set of the resulting component now contains all islands that were part of either one of the components. The new *open edges* are created by merging the *open edges* from both components and then removing those that connect two islands within the new component. The new *open islands* are those islands within the component, that are attached to the new *open edges*.

When on the other side a deduction leads us to *unset* a bridge, we also remove it from all bridge sets it is part of, but we do not change a  $dec\_num$ , as the number of missing

bridges stays the same. If the bridge was stored as an *open edge* for some component, it is removed from this list, and the *open islands* are updated accordingly.

Only bridges with the status *undecided* can be *set* or *unset*. If a deduction attempts to change the status of a bridge that already has a status, no changes are made, and the bridge keeps its current status.

## 6.2 Algorithm

With these principles in place, we turn to the algorithm. The solver iteratively applies deduction rules and complements them with backtracking when no further deductions can be made. We first describe the overall structure of the algorithm before we introduce the individual deduction rules. The structure of the algorithm is illustrated in Figure 8.

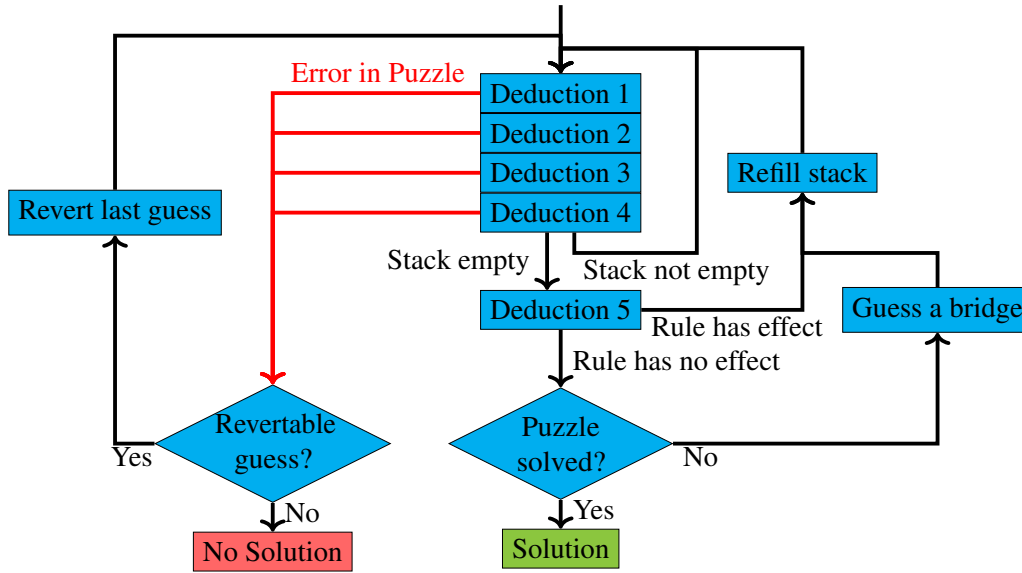


Figure 8: Overview of the algorithm that complements deduction rules with backtracking.

In the first part of the algorithm, we go through the islands and iteratively try to apply deduction rules 1 to 4. These first four deduction rules work locally on one specific island and its bridge sets. To ensure that all islands are covered, we fill a stack with all islands in the puzzle. The island on top of the stack is always the next to be inspected. It is popped from the stack, and we try to apply the four deduction rules one after another. Whenever we can make a deduction and *set* or *unset* some bridge, the two islands connected to this bridge change. As these two islands have just changed, the likelihood of other deductions being applicable to them afterwards is increased. For this reason, they are pushed on top of the stack so that they are inspected next. For this, we accept that islands may be on the stack multiple times. Checking for each island whether it is already on the stack requires more time than applying the four reduction rules. This approach of pushing updated islands to a stack leads to a solving behaviour similar to how humans would solve this puzzle: When they find a new deduction in some region, they continue looking in the same region,

as this clue could lead to a chain of other deductions. This process of applying deductions to the top stack element is repeated until the stack is empty.

When the stack gets empty, this means that none of the first four deduction rules lead to any new deductions for any of the islands. Here, the fifth deduction rule comes in. In contrast to the first deduction rules, this last rule does not look at individual islands, but more globally at components using Hashiwokakero Rule 6. For this, it uses the component sets that we constantly update when *setting* or *unsetting* a bridge. If this deduction rule leads to new deductions, the stack is refilled with all islands in the puzzle and the changed islands are pushed to the top. Then we restart the iteration of deduction rules 1 to 4. This process is repeated until deduction rule 5 also does not lead to changes anymore. Now, if the puzzle is solved, then we are done. If not, then we need to switch to a new strategy, as deduction rules 1 to 5 do not lead to any changes. Whenever this happens, the algorithm uses a backtracking strategy to solve the puzzle.

For this, we first store the current state of the puzzle, which means the bridge sets for each island and the collision sets, as well as the component sets. We then take some *undecided* bridge and *set* it, thus changing its two island endpoints. With this new information, we refill the stack with all islands and proceed with deductions 1 to 5 as described before. As we do not know whether the guess was right, we need to check for errors. To do so, after each deduction rule that leads to some deduction, we do failure checks.

As Hashiwokakero rules 1,2 and 4 are already fulfilled by our definition of islands and bridges and rule 3 is always fulfilled by our handling of colliding bridges, the only errors that can happen are violations of Hashiwokakero Rules 5 or 6. For each of these two rules, we do a failure check.

First, whether the bridge constraint required by Hashiwokakero Rule 5 is still achievable. There are two ways Rule 5 can be violated. Either there are already too many bridges *set*, or there are not enough bridges left to be *set*. So if the *decreased bridge constraint* is lower than 0, this means that we have *set* more bridge than allowed, and we found an error. If, on the other side, the *decreased bridge constraint* is bigger than the number of bridges in the bridge set for this island, then there are not enough *undecided* bridges left to fulfil the constraint, and we have also found an error. Both of these errors are due to a violation of Hashiwokakero Rule 5.

Second, we also check whether full connectivity of the puzzle, as required by Hashiwokakero Rule 6, is still possible. For this, we go through all components of the puzzle. When there exists more than a single component in the puzzle and some component has no bridge in *open edges*, then this component cannot be connected to any other component. This means that all islands within the component are not connected to islands in the other components, and thus, Hashiwokakero Rule 6 of full connectivity is violated.

If both failure checks are passed, the algorithm continues with the next deduction rules and islands. Deduction rules 1 to 5 are repeated in the context of the last guess, until another guess becomes necessary, an error is detected or a solution is found.

If another guess is necessary, we again store the current state of the puzzle, *set* some *undecided* bridge and continue as described above. This whole process of repeating deduction rules 1 to 5 and guessing is repeated until an error is detected, or a solution is found.

Whenever an error is detected, the solver does a rollback to the last state that has been stored. If there is no last state, then there is no more solution possible and we are done. As we have now learned that *setting* the last guessed bridge causes an error, we now know for sure that this bridge has to be *unset* in the context of the previously made decisions. This is now not a guess anymore, but a reliable fact within the context, so we do not need to store the current state. With this new information, we continue. We refill the stack with all islands and continue with the iteration of deduction rules 1 to 4 as described above.

When there is no *undecided* bridge left, and we have not yet found an error, we have found a solution candidate. This solution consists of all bridges that are currently *set*. The solution candidate is verified by checking whether all decreased bridge constraints are 0 and whether there is only one component left, which means that all islands are connected. If the candidate fulfils these rules, all *set* bridges together form a solution to Hashiwokakero. If the puzzle is unique, or we only wanted to find a single solution, we are done now.

If we want to find all solutions in a puzzle that has multiple solutions, we store the solution in a list and proceed with a rollback and remove the last bridge that was *set*, just like if an error had been found. We then refill the stack and continue with deduction rules 1 to 5. This process is repeated until there is no decision left to be reverted. If that is the case, then we are done. The solutions that have been stored up to this point are an exhaustive list of all solutions that are possible for this Hashiwokakero instance.

### 6.3 Deduction Rules

Having explained the structure of the algorithm, we now turn to the deduction rules that are used in the algorithm. The deduction rules were selected by manually finding some applicable deduction, generalising it to a rule and adding it to the solver. Then, the puzzle was solved with the algorithm until no further deductions were possible, after which we reviewed the current state of the puzzle and manually found another new deduction. This process was repeated until the algorithm could solve all the tested puzzles. Then we analysed whether some of the deduction rules were already implied in others, or whether some of them could be generalised. The remaining set of deduction rules has proven to be a really fast and efficient way to solve the Hashiwokakero puzzle during the evaluation presented in Chapter 7. The deduction rules will be described now.

**Deduction Rule 1** Whenever an island has a *decreased bridge constraint* of 1, then only one more bridge is allowed to be *set*. So whenever there are two *undecided* bridges to the

same *neighbouring island*, we know that the second of them can never be *set*, as this would imply that the first bridge is also *set*. But this is not possible without violating the bridge constraint. Thus, the second bridges in directions with two *undecided* bridges can be *unset*.

---

#### Deduction Rule 1

---

```

1: function RULE1(island a)
2:   if dec_num(a) == 1 then
3:     for all n ∈ neighbours(a) do
4:       if bridgesan = {Ban1, Ban2} then
5:         Unset Ban2.

```

---

**Deduction Rule 2** Whenever the number of *undecided* bridges for an island equals the *decreased bridge constraint* of the island, all these *undecided* bridges can be *set*.

---

#### Deduction Rule 2

---

```

1: function RULE2(island a)
2:   if dec_num(a) == |bridgesa| and dec_num(a) ≠ 0 then
3:     for all n ∈ neighbours(a) do
4:       Set all bridges in bridgesan.

```

---

**Deduction Rule 3** Whenever the *decreased bridge constraint* of an island equals 0, while the bridge sets of the island are not empty, all remaining *undecided* bridges connected to this island can be *unset*.

---

#### Deduction Rule 3

---

```

1: function RULE3(island a)
2:   if dec_num(a) == 0 and |bridgesa| > 0 then
3:     for all n ∈ neighbours(a) do
4:       Unset all bridges in bridgesan.

```

---

**Deduction Rule 4** The next deduction rule allows us to derive in certain constellations that some bridges can be *set*. Whenever the *decreased bridge constraint* of an island is exactly one lower than the number of bridges in its bridge sets, then the first bridge in each bridge set with two *undecided* bridges can be *set*. As we know that all but one *undecided* bridge will be *set*, we can certainly know, that this *unset* bridge will be some second bridge, as a first bridge can only be *unset* when the corresponding second bridge is *unset*.

---

#### Deduction Rule 4

---

```

1: function RULE4(island a)
2:   if dec_num(a) == |bridgesa| - 1 then
3:     for all n ∈ neighbours(a) do
4:       if bridgesan = {Ban1, Ban2} then
5:         Set Ban1.

```

---

**Deduction Rule 5** While deduction rules 1 to 4 work locally on individual islands, deduction rule 5 takes a global perspective by considering components in the puzzle. According to Hashiwokakero Rule 6, all islands must be connected, which means that all of them must belong to the same component. The idea of this deduction rule is to *unset* all bridges that, if *set*, would connect two components in a way that isolates the resulting component from the other components in the puzzle.

The rule is only applicable when more than two components exist, as the last two components are allowed to be connected, considering that they contain all islands in the puzzle. We then iterate over all components in the puzzle. For each component, we count the *open islands*. As defined in Definition 6.3, *open islands* are islands with *undecided* bridges that can connect this component to another component. If a component contains only one open island, this island is a candidate for the deduction: For each *undecided* bridge connected to this island, we examine the island on the other side. If this island is also the only *open island* of its component, then *setting* bridges between those islands could cause a problem: if both their bridge constraints are satisfied by this connection, the resulting component has no open islands left and is thus isolated from the rest of the puzzle. This situation can only occur if both islands have a *decreased bridge constraint* of either one or two. In the case where both *decreased bridge constraints* are one, all *undecided* bridges between the two islands can be *unset*, as *setting* a bridge would isolate the resulting component. In the case where both *decreased bridge constraints* are two, the second bridge between the islands can be *unset*. *Setting* one bridge between these islands would be possible without violating a rule, but *setting* both bridges would again isolate the resulting component. This process is repeated for each component, and all bridges that could isolate components are *unset*. Formally, we implement the following deduction rule.

---

**Deduction Rule 5**

---

```

1: function RULE5()
2:   if |components| > 2 then
3:     for all  $c_k \in \text{components}$  do
4:       if  $\text{open\_islands}_{c_k} == \{a\}$  for some island  $a \in \text{islands}_{c_k}$  then
5:         for all  $B_{ac}^i \in \text{open\_edges}_{c_k}$  do
6:           if  $\text{open\_islands}_{\text{component}(c)} == \{c\}$  then
7:             if  $\text{dec\_num}(a) == 1$  and  $\text{dec\_num}(c) == 1$  then
8:               Unset all bridges in  $\text{bridges}_{ac}$ .
9:             else if  $\text{dec\_num}(a) == 2$  and  $\text{dec\_num}(c) == 2$  then
10:              Unset  $B_{ac}^2$ .

```

---

With these deduction rules and the complementing backtracking strategy, we have implemented a solver for Hashiwokakero. The local deduction rules solve the puzzle incrementally, making deductions similar to human reasoning. The more complex, global, and connectivity-based deduction rule 5 is only used when further deductions cannot be reached locally. Whenever the deduction rules alone are insufficient, backtracking



systematically explores the remaining possibilities. Together, these techniques allow the solver to efficiently handle both uniquely solvable instances and instances with multiple solutions.

## 7 Evaluation

In this chapter, we evaluate the performance of the solving approaches that we implemented in Chapters 4-6. We analyse how they scale with increasing island numbers and how they perform on unique puzzles and puzzles with multiple solutions. We compare the approaches and discuss possible reasons for differences in performance.

For the benchmark, we used two test suites with Hashiwokakero instances. The first one is a subset of the test suite by Coelho et al. [6]. However, since this dataset contains neither small puzzles nor a sufficient number of puzzles with a unique solution, we additionally implemented an own generator to produce two further test suites, one for unique puzzles and another one for puzzles with multiple solutions. This generator is described now.

### 7.1 Generator

We implement a generator that produces puzzles with a specified number of islands. Additionally, it can be specified whether the generated puzzles should be unique, as is typical in puzzle books, or have multiple solutions, which is computationally interesting. The generator is designed to produce puzzles that resemble instances presented in puzzle books.

The process of puzzle generation involves several steps. First, an empty puzzle is initialised, and a generation strategy is selected. For most puzzles, we use the basic generation strategy. However, this approach is inefficient for large, unique puzzles, so a different strategy is adopted for these. After this generation step, any empty rows or columns at the borders are removed before the puzzle is saved. The two generation strategies are described in the following.

**Basic generation strategy** The basic generation strategy takes a specified number of islands and a property as input. The property *unique* requires that the puzzle has exactly one solution and the property *multiple solutions* requires at least two solutions. For this basic generation strategy, we take a similar approach as Coelho et al. [6], but we adapt the approach in order to generate puzzles that more closely resemble those commonly found in puzzle books.

The algorithm begins by placing an island at a random position. It then repeats the following steps: a random existing island is selected, together with a random direction (up, down, left, or right). A random bridge length is chosen between a minimum length of 2 and the distance to the border. If the resulting bridge does not collide with existing bridges or islands, it is inserted, together with a new island at its endpoint. With a probability of 50%, the bridge is chosen to be either single or double. The bridge constraint of both connected islands is increased accordingly. This process is repeated until either the specified number of islands is reached, or the predefined attempt limit is exceeded. In this case, the instance

is discarded and regenerated.

The solution of the resulting puzzle instance is a tree, as it contains no cycles. These are added in the next step. Now all pairs of neighbouring islands without a bridge between them are listed, and a bridge is added between each pair with a probability of 50%. Again, the type of bridge (single or double) is selected randomly with a probability of 50%.

After adding the cycles, the property of the puzzle is checked against the specified property. This is done by first trying to find the property using the deduction-based approach with a time limit of 5 seconds and complementing this strategy with the SMT-based reachability approach with a time limit of 300 seconds. This choice of solvers will be explained in the discussion of evaluation results in Section 7.4. For both solvers, we implemented a specialised function, that tries to find up to two solutions and deduces from the result whether the puzzle is *unique* or has *multiple solutions*. If the property of the puzzle matches the specified property, the generation is complete. Otherwise, the added cycles are removed, and up to four more attempts are made to insert cycles in different positions, again randomised with a probability of 50% per bridge, to see if the property is correct. If none of these attempts succeeds, the puzzle is discarded and regenerated. This process is repeated until either a valid puzzle instance is produced or the predefined limit of attempts is reached.

**Unique generation strategy** The basic generation strategy of randomly generating an instance followed by a final uniqueness check is inefficient for large, unique puzzles. To address this problem, we use a different strategy for unique puzzles with more than 80 islands. This strategy enforces uniqueness incrementally throughout the generation process.

First, we generate a unique puzzle with 80 islands using the basic generation strategy. This puzzle is then iteratively extended by adding islands in batches of ten. After each extension, we verify that the puzzle is still uniquely solvable; if not, the extension is discarded. Then, another batch is added. This process is repeated until the desired number of islands is reached. If a predefined number of attempts fail, the entire process is repeated by generating a new instance with 80 islands using the basic generation strategy. This incremental approach significantly improves the efficiency for generating large unique puzzles while maintaining the uniqueness constraint. It is important to note that we do not add additional cycles in unique puzzles after the first 80 islands, as this likely adds additional solutions.

Note that instances generated by some generator do not necessarily represent those of other generators or published puzzle books. Since structural differences in the generated puzzles can influence solver performance, we will complement our generated instances with a test suite from the literature.

## 7.2 Benchmark Setup

Having a means to generate a test suite, we now introduce setup and design of the benchmark. All solving approaches are implemented in Python 3, except for the underlying SAT and SMT solvers, which are implemented in C++. We used the SAT solver *Glucose 3.0* [23], through its *PySAT* interface and the SMT solver *Z3* [21], via its Python API. The benchmark was conducted on a server with an AMD EPYC 9354P CPU (32 cores / 64 threads, 3.8Ghz), 512 GB DDR5-4800 DRAM, running Ubuntu 24.04.1 LTS with Python version 3.12.

We conducted a benchmark for five solving approaches: The two SAT-based approaches *HashiSAT-Reachability* and *HashiSAT-Mixed-Approach*, the two SMT-based approaches *HashiSMT-Reachability*, *HashiSMT-Mixed-Approach*, and the deduction-based *Hashi-Solver*. The approach *HashiSAT-Brute-Force* was excluded from the benchmark, as it performed very poorly in preliminary tests on small puzzle instances, and it can be expected to have a bad scalability to larger instances.

We benchmark the solvers on two generated test suites and on a subset of the test suite by Coelho et al. [6]. As the composition of these test suites is relevant for the benchmark results, we shortly describe them.

We generated two test suites, one for unique puzzles and one for puzzles with multiple solutions using the generator described in Section 7.1. As we were interested both in small puzzles, that might be in puzzle books, but also in large puzzles, that are computationally interesting, we decided to benchmark puzzles for every island number up to 60, and then continue in steps of 10, to ensure a reasonable runtime for the whole benchmark. For each island number, we included five instances and used the average results to reduce the impact of individual puzzle properties. While this number cannot compensate for all individual puzzle properties, it allows us to see trends for increasing islands while being feasible in terms of overall runtime.

The test suite for puzzles with a unique solution contains a total of 465 puzzles with five puzzles for each island number in  $\{2, 3, 4, \dots, 59\}$  and  $\{60, 70, 80, \dots, 400\}$ . The test suite with multiple solutions starts from an island number of four, as there are no puzzles with multiple solutions for fewer than four islands. The test suite for multiple solutions thus contains a total of 455 puzzles with five puzzles for each island number in  $\{4, 5, 6, \dots, 59\}$  and  $\{60, 70, 80, \dots, 400\}$ .

The test suite by Coelho et al. [6] contains 1440 puzzles. They introduce a parameter  $\alpha$ , that influences the number of cycles in the puzzle, and a parameter  $\beta$ , that influences the number of double bridges. The test suite contains puzzles with the island numbers 100, 200, 300 and 400. For each combination of island number,  $\alpha$  and  $\beta$ , there are 30 puzzles in the test suite. To ensure a feasible runtime while including puzzles with various parameter settings, we decided to include only the first 5 of the 30 puzzles (i.e. ids 001-005 as specified in the file names) for each combination of island number,  $\alpha$  and  $\beta$ . This results

in a total test suite size of 240 puzzles, with 60 puzzles per island number.

For puzzles with a unique solution, we benchmarked the time required by each solver to find that one solution. For puzzles with multiple solutions, we considered two tasks: First, finding one valid solution, and second, computing all valid solutions.

To ensure a feasible overall runtime for the benchmark, we set a time limit of 180 seconds per solver and instance. If this limit is exceeded, the run is recorded as a timeout, and it is included in average calculations with a time of 180 seconds.

Each instance is solved three times by each solver and the median time is stored to account for nondeterministic behaviour and to reduce the influence of external factors. If a solver times out in any run, we do not perform further repetitions for that instance, and directly store the timeout to keep the benchmark feasible.

### 7.3 Results

We visualise the benchmark results using line graphs and SV-Comp-style plots [24]. To properly interpret these plots, we briefly clarify how SV-Comp-style plots can be interpreted and which puzzles are illustrated in line graphs and SV-Comp-style plots, respectively.

We start with introducing SV-Comp-style plots. The x-axis of an SV-Comp-style plot shows the cumulative scores for each solver, where a solver receives one score point for each puzzle for which it returns the expected solution. The y-axis displays the time on a logarithmic scale. Note that the time is *not* cumulative. Each data point represents a cumulative score and the time at which this score was reached. The SV-Comp-style plot is interpreted as follows: If there is a data point with score 100 at a time of 10 seconds, then this means that the corresponding solver returned the expected solution to 100 puzzles within 10 seconds. Thus, the farther to the right a line goes for some solver, the more puzzles it has solved, and the lower the line is for some score, the faster it has solved the number of puzzles represented by this score. Furthermore, note that solvers with the same cumulative score have solved the same number of puzzles within this time, but not necessarily the same instances. Thus, the SV-Comp-style plot only allows us to conclude how solvers perform on a whole test suite, but not how they perform on any individual instance.

We now describe, which puzzles are included in line graphs and SV-Comp-style plots. For puzzles in our generated test suites, we always first show a diagram with the result of small puzzles with up to 60 islands. In these diagrams the average of five puzzles is displayed for each island number in  $\{2, 3, \dots, 60\}$  for unique puzzles and in  $\{4, 5, \dots, 60\}$  for puzzles with multiple solutions. We then show a diagram that illustrates the average performance of five puzzles for each island number in  $\{10, 20, 30, \dots, 400\}$ . For the multiple solution test suite, these puzzles with island numbers  $\{10, 20, 30, \dots, 400\}$ , are additionally illustrated in an SV-Comp-style plot. For the test suite by Coelho et al. [6], we present one line graph to display the average results for 60 puzzles for the island numbers in  $\{100, 200, 300, 400\}$ .

The same puzzles are also illustrated in an SV-Comp-style plot. Having explained this, we start with evaluating the results for puzzles with a unique solution.

### 7.3.1 Puzzles with a Unique Solution

The benchmark results for solving small, uniquely solvable puzzles with 2 to 60 islands are illustrated in Figure 9. HashiSolver consistently achieves the best performance, closely followed by HashiSAT-Mixed-Approach. The two SMT-based approaches, HashiSMT-Mixed-Approach and HashiSMT-Reachability, perform second and third-best overall, respectively. HashiSMT-Mixed-Approach is the faster of the two, but shows significant fluctuations. HashiSAT-Reachability performs better than the SMT-based approaches for very small puzzles, but its runtime increases more quickly with growing island number than that of all other solvers.

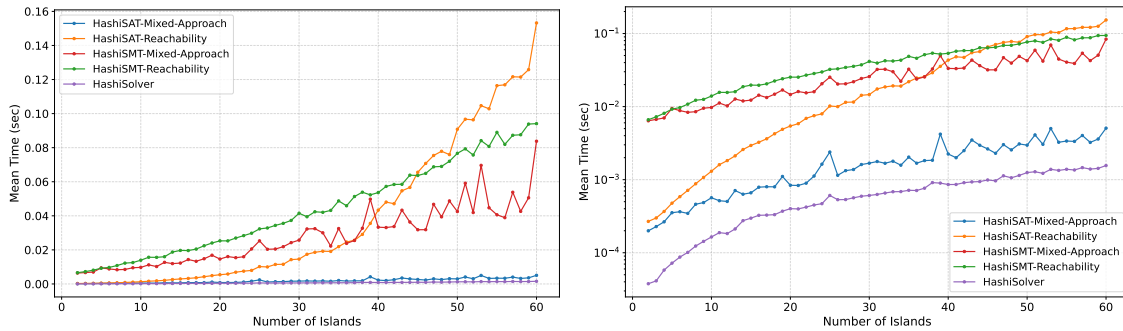


Figure 9: Benchmark results for finding the solution to uniquely solvable puzzle instances with 2 to 60 islands. Each data point represents the average time of five instances. The left plot uses linear scale for the y-axis, while the right plot uses logarithmic scale.

For solving large instances, as illustrated by Figure 10, HashiSolver remains the best approach. HashiSAT-Mixed-Approach is still second best, but the gap to the SMT-based approaches becomes less when the island number increases. The SMT-based approaches show similar performance characteristics as for smaller puzzles. Interestingly, the fluctuations of the SAT- and SMT-based mixed approaches show very similar patterns in their fluctuations. HashiSAT-Reachability performs worst overall. Its runtime increases quickly with growing island numbers, and it fails to solve four instances with big island numbers within the time limit.

### 7.3.2 Puzzles with Multiple Solutions

For puzzles with multiple solutions, two tasks were benchmarked per solver: Finding one solution and finding all solutions for a puzzle. We first present the results for finding one solution.

**Finding One Solution** For finding one solution to small puzzles with up to 60 islands and multiple solutions as illustrated by Figure 11, HashiSolver again consistently performs best,

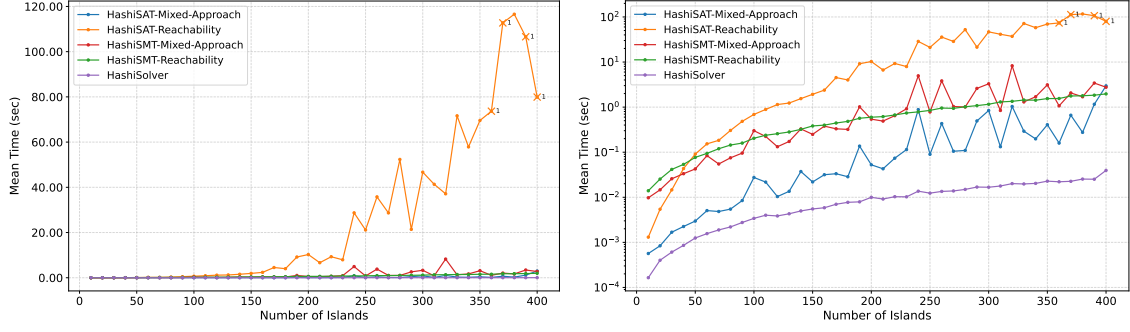


Figure 10: Benchmark results for finding the solution to uniquely solvable puzzle instances with 10 to 400 islands (in steps of 10). Each data point represents the average time of five instances. Timeouts are marked by an X labelled with the number of timed-out instances. Timeouts are counted as 180 seconds in the calculations of averages. The left plot uses linear scale for the y-axis, while the right plot uses logarithmic scale.

closely followed by HashiSAT-Mixed-Approach. The two SMT-based approaches perform similar compared to before for small unique puzzles (cf. Figure 9), with HashiSMT-Mixed-Approach being faster on average but also having significantly more fluctuations. As before, HashiSAT-Reachability starts off fast but slows down quickly when the island number increases.

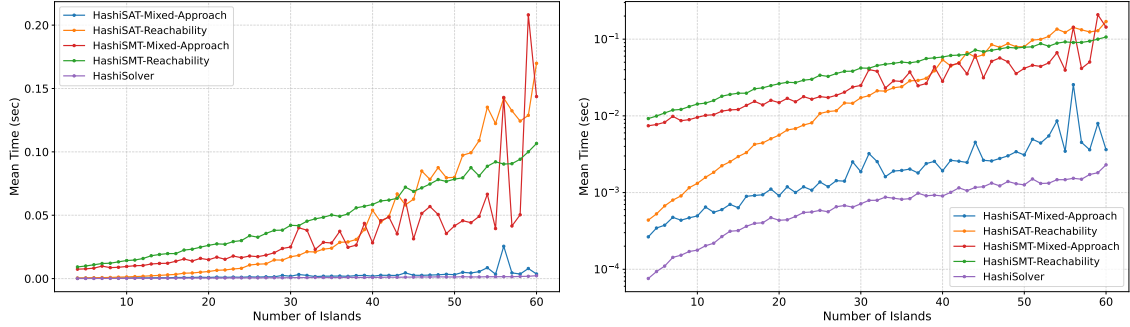


Figure 11: Benchmark results for finding one solution to puzzle instances with 4 to 60 islands and multiple solutions. Each data point represents the average time of five instances. The left plot uses linear scale for the y-axis, while the right plot uses logarithmic scale.

For finding one solution to large puzzles with up to 400 islands and multiple solutions, as shown in Figure 12, HashiSolver remains the best-performing approach for most inputs, although it shows few significant peaks, including two cases where it exceeds the time limit. Interestingly, the timeout on one of the five puzzles with 120 islands is not shared by the other solvers. The timeout for one puzzle with 290 islands is, however, evident for all solvers, except HashiSMT-Reachability. The performance of the solvers relative to one another is similar compared to the larger puzzles with a unique solution (cf. Figure 10, but note that the linear y-axes are scaled differently). However, all solvers, except for HashiSMT-Reachability, fluctuate more severely than before and HashiSAT-Reachability performs a lot worse than before.

The SV-Comp-style plot in Figure 13 illustrates the solver performances for finding one

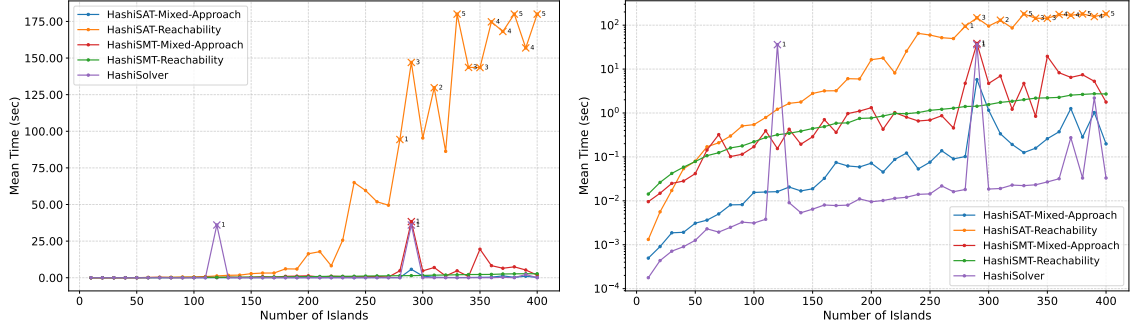


Figure 12: Benchmark results for finding one solution to puzzles instances with 10 to 400 islands (in steps of 10) and multiple solutions. Each data point represents the average time of five instances. Timeouts are marked by an X labelled with the number of timed-out instances. Timeouts are counted as 180 seconds in the calculations of averages. The left plot uses linear scale for the y-axis, while the right plot uses logarithmic scale.

solution to the same 200 puzzle instances as visualised in Figure 12. HashiSolver is fast for most puzzles, but has a hard time with a few. Overall, it solves two puzzles less than the second-fastest solver HashiSAT-Mixed-Approach. HashiSMT-Reachability is the most consistent approach. It is the fastest solver to solve all 200 instances, even though there are only few puzzles where it is faster than HashiSolver and HashiSAT-Mixed-Approach. HashiSMT-Mixed-Approach shows a similar curve as its SAT counterpart, but it is always slower. As was already apparent from the previous diagrams, HashiSAT-Reachability is by far the worst-performing solver.

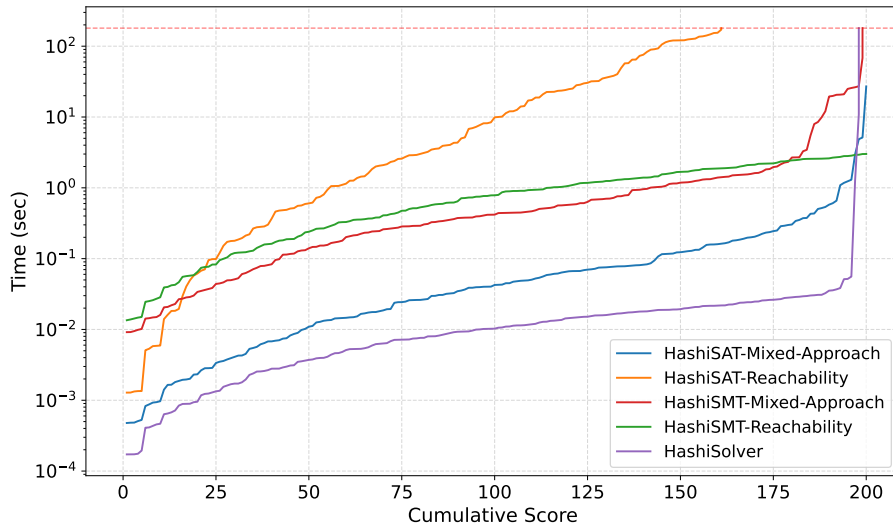


Figure 13: Benchmark results for finding one solution to 200 puzzle instances with multiple solutions in an SV-Comp-style plot [24]. The 200 puzzle instances contain puzzles from 10 to 400 islands in steps of 10, with 5 instances each.

**Finding All Solutions** Figure 14 shows the results for finding all solutions to puzzles with 4 to 60 islands and multiple solutions. All solvers show similar patterns of peaks and fluctuations. HashiSolver performs best overall, outperforming all other approaches for



54 of the 60 categories, but it is also the only solver that reaches the time limit for some instance. HashiSAT-Mixed-Approach consistently ranks second. As before, HashiSAT-Reachability starts off fast but slows down even more quickly for increasing island numbers than before. The SMT-based approaches perform similarly to one another. They start with an overhead, but improve relative to the other solvers as island numbers increase.

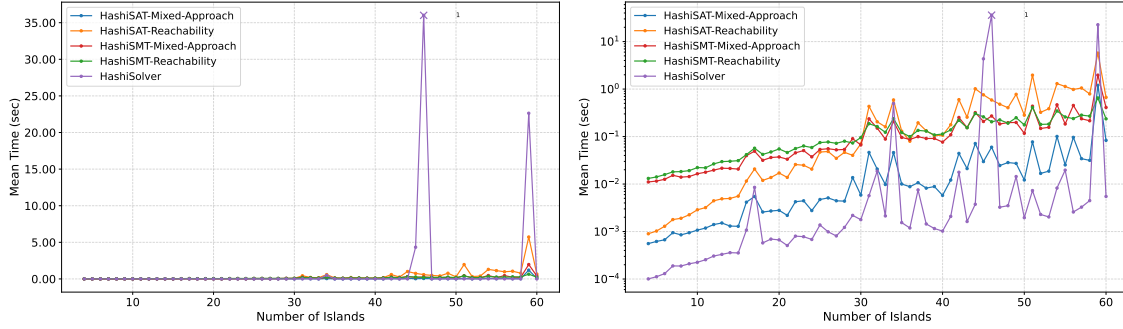


Figure 14: Benchmark results for finding all solutions to puzzle instances with 4 to 60 islands and multiple solutions. Each data point represents the average time of five instances. Timeouts are marked by an X labelled with the number of timed-out instances. Timeouts are counted as 180 seconds in the calculations of averages. The left plot uses linear scale for the y-axis, while the right plot uses logarithmic scale.

For large puzzles with up to 400 islands, as illustrated by Figure 15, again, all solvers show similar fluctuation patterns. HashiSolver performs best for some island numbers, but it is outperformed by most other solvers for other island numbers. The two mixed approaches show a similar performance to one another with smaller fluctuations than HashiSolver. HashiSMT-Reachability performs slightly better than the mixed approaches with even smaller fluctuations. HashiSAT-Reachability performs worst and reaches the time limit for all instances with more than 230 islands.

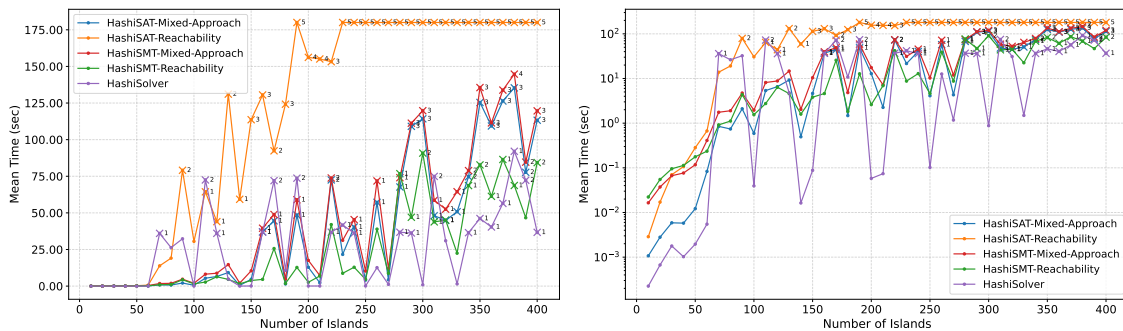


Figure 15: Benchmark results for finding all solutions to puzzle instances with 10 to 400 islands (in steps of 10) and multiple solutions. Each data point represents the average time of five instances. Timeouts are marked by an X labelled with the number of timed-out instances. Timeouts are counted as 180 seconds in the calculations of averages. The left plot uses linear scale for the y-axis, while the right plot uses logarithmic scale.

The SV-Comp-style plot in Figure 16 illustrates the solver performances for finding all solutions to the same 200 puzzles instances as visualised in Figure 15. This plot allows us

to compare the solver performance more clearly without fluctuations that make the diagram difficult to read. HashiSolver solves most of the puzzles quickly, but does not solve as many puzzles as HashiSMT-Reachability within the time limit. The two mixed approaches solve almost the same number of puzzles within the time limit, though HashiSAT-Mixed-Approach within the time limit, though the SAT-based mixed approach solves many puzzles faster. Consistent with the previous results, HashiSAT-Reachability performs worst. In contrast to finding one solution (cf. Figure 13), no solver is able to finish the task of finding all solutions within the time limit.

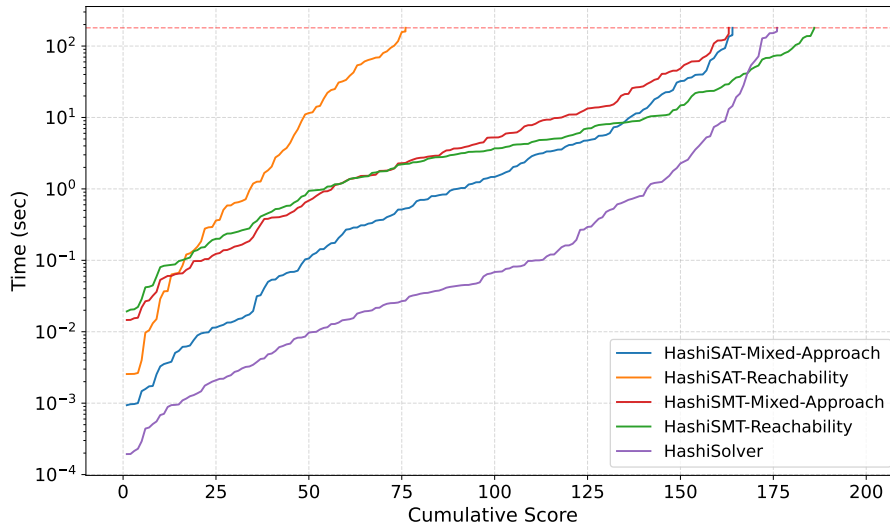


Figure 16: Benchmark results for finding all solutions to 200 puzzle instances with multiple solutions in an SV-Comp-style plot [24]. The 200 puzzle instances contain puzzles from 10 to 400 islands in steps of 10, with 5 instances each.

### 7.3.3 Puzzles from Literature

We now evaluate the benchmark results from the test suite by Coelho et al. [6]. As explained in 7.2, we use a subset of the test suite they provided. As before, we benchmarked two tasks for each solver: Finding one correct solution and finding all correct solutions for a puzzle. We first present the results for finding one solution.

**Finding One Solution** Figure 17 shows the results for the task of finding one solution. For instances with 100 islands, HashiSMT-Reachability performs best, closely followed by HashiSAT-Reachability. They are the only solvers that solve all instances with 100 islands within the time limit. They are followed by HashiSolver with 8 timeouts, HashiSAT-Mixed-Approach with 16 timeouts, and HashiSMT-Mixed-Approach with 20 timeouts.

For instances with 200 islands, HashiSMT-Reachability remains the best-performing solver. HashiSAT-Reachability keeps the second place, although the performance gap between the two approaches increases significantly. Nevertheless, they remain the only solvers without timeouts in this category. They are again followed by HashiSolver and

HashiSAT-Mixed-Approach, both with 24 timeouts, and HashiSMT-Mixed-Approach with 30 timeouts.

For instances with 300 islands, the ranking changes. HashiSMT-Reachability continues to perform best overall as the only solver solving all instances, while HashiSAT-Reachability drops from second to last place in terms of average runtime. Behind HashiSMT-Reachability follow HashiSAT-Mixed-Approach with 31 timeouts, HashiSolver with 35 timeouts, and HashiSMT-Mixed-Approach with 39 timeouts. HashiSAT-Reachability finishes ranked last, as its average solving time is the highest, despite having only 32 timeouts.

The ranking remains similar for instances with 400 islands. HashiSMT-Reachability again performs best by a large margin still being able to solve all instances. It is followed by HashiSAT-Mixed-Approach with 38 timeouts. HashiSMT-Mixed-Approach and HashiSolver have a similar runtime with 41 and 43 timeouts. HashiSAT-Reachability performs worst overall, reaching the time limit for 59 of the 60 instances in this category.

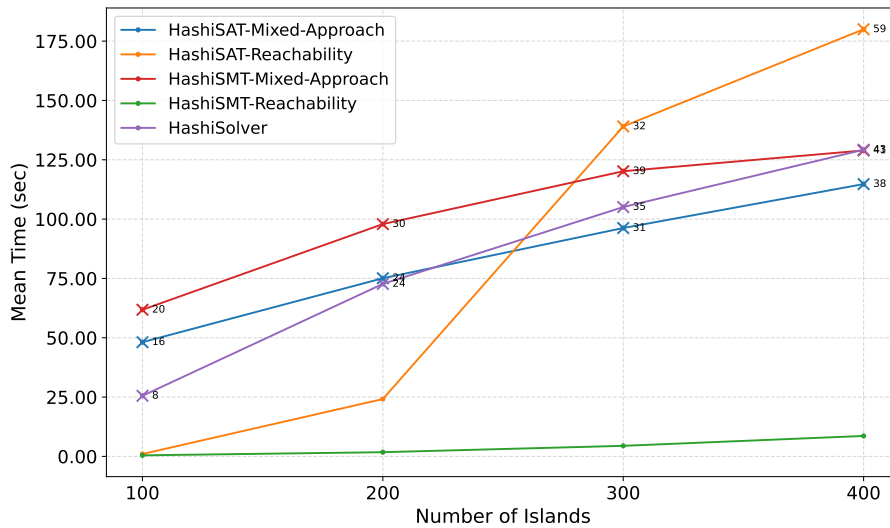


Figure 17: Benchmark results for finding one solution to puzzle instances with 100, 200, 300 and 400 islands from the benchmark suite by Coelho et al. [6]. Each data point represents the average time of 60 instances.

The SV-Comp-style plot in Figure 18 illustrates the solver performances for finding one solution to the same 240 puzzles instances as visualised in Figure 17. The plot confirms that HashiSMT-Reachability is by far the most reliable solver for puzzles from the test suite by Coelho et al. [6]. It consistently solves instances and eventually outperforms all other approaches by a large margin, being the only solver capable of solving all 240 instances within the time limit. Even though HashiSolver and HashiSAT-Mixed-Approach are the fastest and second-fastest approach for nearly half of the instances, they reach a time limit for the other half of the instances. HashiSMT-Mixed-Approach performs worst in terms of solved instances and performance. Interestingly, HashiSAT-Reachability solves

nearly 150 instances and is thus the second-best solver in terms of solved instances, even though it is the slowest solver in the beginning.

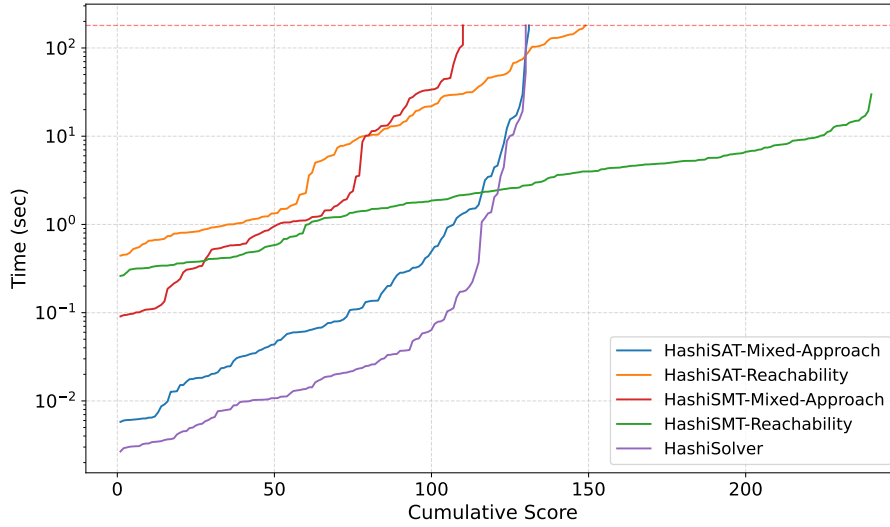


Figure 18: Benchmark results for finding one solution to 240 puzzle instances in an SV-Comp-style plot [24]. The 240 puzzle instances contain 60 instances for each of the island numbers 100, 200, 300 and 400 from the benchmark suite by Coelho et al. [6].

**Finding All Solutions** Figure 19 shows the results for the task of finding all solutions. For instances with 100 islands, HashiSMT-Reachability performs best overall, reaching a timeout for 21 of 60 instances. It is followed by HashiSolver with 30 timeouts. HashiSAT-Mixed-Approach and HashiSMT-Mixed-Approach perform similarly, both timing out on 38 instances. HashiSAT-Reachability performs worst in this category with the highest average runtime and 38 timeouts.

For instances with 200 islands, the number of timeouts increases. HashiSolver becomes the best-performing approach, timing out on 50 of the 60 instances. It is followed by HashiSMT-Reachability, with 56 timeouts. The two mixed approaches each have 58 timeouts, while HashiSAT-Reachability fails to solve any instance within the time limit. This trend continues for instances with 300 islands, where HashiSolver is the only approach capable of solving a single instance within the time limit. All other solvers time out on every instance. For instances with 400 islands, all solvers time out on every instance.

The SV-Comp-style plot, presented in Figure 20, shows a similar picture as Figure 19. Within the time limit of 180 seconds, no solver finds all solutions for more than 43 instances. It is interesting to note that HashiSMT-Reachability solves the most instances, closely followed by HashiSolver, which is the fastest approach for most instances. The mixed approaches solve the exact same number of instances ranking third and fourth, while HashiSAT-Reachability has the worst performance in terms of running time and solved instances. Overall, the solvers perform much worse for the test suite from Coelho et al. [6], than they do for our test suite (cf. Figure 16). This is an interesting result, that we will

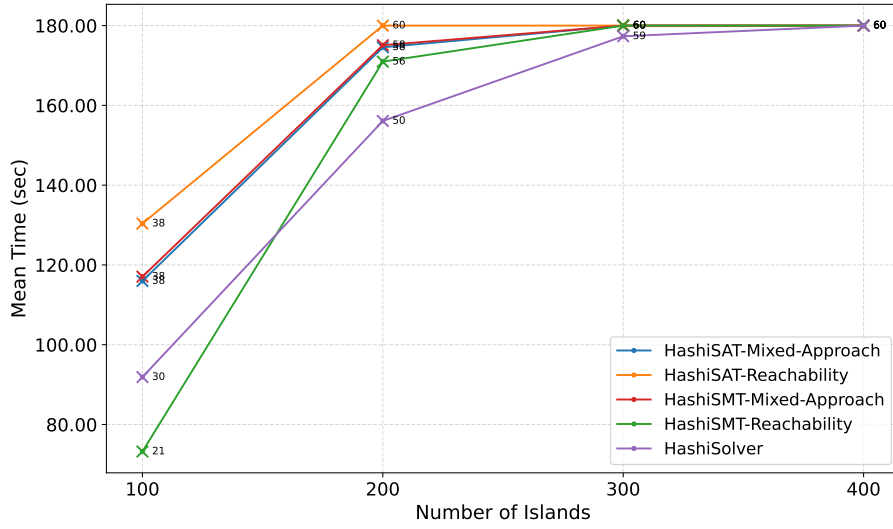


Figure 19: Benchmark results for finding all solutions to puzzle instances with 100, 200, 300 and 400 islands from the benchmark suite by Coelho et al. [6]. Each data point represents the average time of 60 instances.

discuss in the next section.

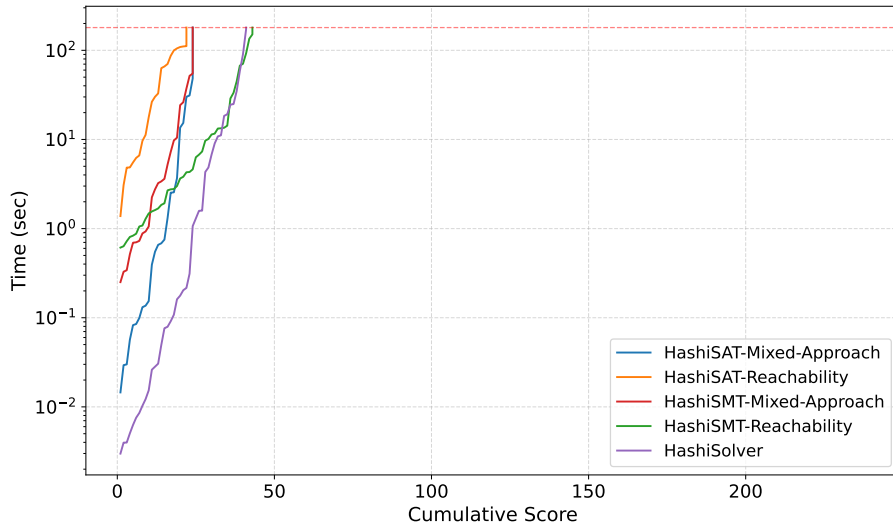


Figure 20: Benchmark results for finding all solutions to 240 puzzle instances in an SV-Comp-style plot [24]. The 240 puzzle instances contain 60 instances for each of the island numbers 100, 200, 300 and 400 from the benchmark suite by Coelho et al. [6].

## 7.4 Discussion

In this section, we discuss the evaluation results. First, we put the performance of each solver into context, after which we compare the different encodings within the approaches and the differences between test suites.

HashiSolver achieves the best performance on most instances. It is the fastest approach for puzzles with a unique solution and for finding one solution in puzzles with multiple

solutions. For finding all solutions, however, it shows significant fluctuations. While it is still often the best-performing solver, it occasionally reaches the time limit on instances that are solved by other solvers. This behaviour is consistent across both our test suite and the external test suite by Coelho et al. [6].

HashiSAT-Mixed-Approach consistently ranks second after HashiSolver, for both unique puzzles and puzzles with multiple solutions. For finding all solutions in puzzles with multiple solutions, it often has peaks on the same instances as HashiSolver. This suggests that both approaches are affected by similar structural properties and therefore do not complement each other well.

HashiSMT-Reachability starts with an overhead compared to the non-SMT-based solvers. However, its runtime remains stable and scales well with increasing island numbers. While it is among the slowest solvers on small instances, it becomes the most reliable approach on large and difficult instances. This is evident in the test suite by Coelho et al. [6], where HashiSMT-Reachability solves all 240 instances in finding one solution, while the other solvers solve fewer than 180 instances. HashiSMT-Reachability also achieves the best overall performance for finding all solutions in both our test suite and the test suite by Coelho et al. [6]. In conclusion, HashiSMT-Reachability is not suited well for small instances, but it performs well on large and difficult ones. For these reasons, we chose a hybrid strategy in the generator that combines HashiSolver and HashiSMT-Reachability. HashiSolver is used first due to its speed, but after a time limit of 5 seconds, HashiSMT-Reachability is used due to its reliability. With this combination, the property of uniqueness or multiple solutions could efficiently be determined for all generated puzzles.

In our test suite, HashiSMT-Mixed-Approach behaves similarly to HashiSMT-Reachability for most instances, but shows significantly higher fluctuations. On the test suite by Coelho et al. [6], it performs worst among all approaches for finding a single solution and second worst for finding all solutions. So overall, HashiSMT-Reachability performs better and is thus the preferable SMT-based approach.

HashiSAT-Reachability performs well on small instances, but its performance declines significantly when the number of islands increases. This behaviour is expected, since encoding numeric constraints using Boolean variables does not scale efficiently. On the test suite by Coelho et al. [6], it performs second best for finding a single solution for up to 200 islands, being able to solve all instances. But its performance decreases significantly for larger instances. For finding all solutions, it is consistently the worst-performing approach. Overall, HashiSAT-Reachability is only suitable for small instances, while it is outperformed by all other approaches for larger instances.

We now compare the two SMT-based and the two SAT-based approaches. The SMT-based approaches both have an initial overhead. While HashiSMT-Reachability has a stable runtime curve for increasing island numbers, the mixed approach shows significant fluctuations. The SAT-based approaches have a lower overhead than the SMT-based approaches. Apart from that, the SAT-based approaches differ a lot in performance, which

emphasizes the impact of the encoding for Rule 6.

HashiSAT-Mixed-Approach and HashiSMT-Mixed-Approach show similar fluctuation patterns across the benchmarks. These fluctuations are generally more prominent for the SMT-based approach, potentially due to the additional overhead introduced by the theory solvers at the start of each solver invocation. The fact that both mixed approaches show comparable patterns when searching for a single solution suggests that certain puzzle properties affect them more strongly than the other solver variants. One possible factor is the number of candidate solutions satisfying Rules 1-5. A larger number of such candidates can require additional solver calls in the mixed approaches, thereby increasing the overall running time.

For finding all solutions, all solvers show similar fluctuation patterns. In this setting, the fluctuations likely correlate with the total number of valid solutions. SAT- and SMT-based solvers are affected because additional solutions require additional solver invocations, while HashiSolver is impacted through increased backtracking effort during solution enumeration.

The two reachability-based approaches exhibit distinct scaling behaviours. HashiSAT-Reachability performs very well on instances with few islands, but its running time increases rapidly as for growing island numbers. In contrast, HashiSMT-Reachability shows a larger initial overhead, but scales significantly better for larger instances. The SMT solver we use seems to have upfront costs, but can then reason about numerical constraints efficiently. For the SAT solvers, by comparison, we emulate arithmetic with additional constraints and variables. While this approach is efficient for small instances, the number of required constraints grows quickly as the island numbers increase, leading to substantially higher runtimes.

Among the two mixed approaches, HashiSAT-Mixed-Approach consistently outperforms HashiSMT-Mixed-Approach across all benchmarks. This indicates that the primary advantage of SMT in Hashiwokakero lies in handling the global connectivity constraint of Rule 6, and potential benefits of SMT on the local constraints might not outweigh the additional theory solver overhead.

Finally, we compare the results on the suite by Coelho et al. [6] with those from our test suite for multiple solutions. The comparison shows that the instances from Coelho et al. are significantly more difficult for all solvers than those in our test suite. This can be seen for both finding one solution and finding all solutions to the puzzles. For finding one solution, all solvers except HashiSAT-Reachability solved all instances in our test suite in under 10 seconds. In contrast, all solvers except HashiSMT-Reachability could solve only half of the instances in the test suite by Coelho et al. [6] within the time limit. For finding all solutions, the differences are most prominent for puzzles with 300 or 400 islands. In our test suite, all solvers except HashiSAT-Reachability were able to solve several instances within the time limit. In contrast, for the test suite by Coelho et al. [6], only a single instance with 300 islands was solved by HashiSolver within the time limit, while all other



solvers timed out for all instances with 300 or 400 islands. It is not completely clear what causes the differences between the test suites. Coelho et al. have shown in their work that the parameters influencing the number of double bridges and the number of cycles in the puzzle have an influence on the performance of their solver [6]. In our generated test suite, we used fixed values for these parameters. So, some differences in performance can be explained by different parameter settings, but even for the instances with similar parameter settings, the puzzles in our test suite appeared to be significantly easier. Identifying which aspects of the generation process and which structural puzzle properties are responsible for this difference is left as a question for further research.



## 8 Conclusion

In this thesis, we studied the problem of solving Hashiwokakero puzzles from both a theoretical and a practical perspective. After formally defining the puzzle, we discussed its NP-completeness, which motivates the use of solving approaches with SAT and SMT.

Building on this foundation, we developed and analysed three different solvers. First, we introduced a SAT-based solver with three different encodings for the connectivity rule. Second, we presented an SMT-based solver with two encodings for the connectivity rule. And third, we implemented a deduction-based solver that combines human-like reasoning with backtracking. In addition, we designed a puzzle generator capable of producing both uniquely solvable puzzles and puzzles with multiple solutions. Finally, we evaluated the approaches on two test suites. One of these was generated by our own generator the other one was provided by Coelho et al. [6].

The benchmark results show differences between the approaches. The deduction-based HashiSolver achieves the best performance on most instances, while the SMT-based reachability approach is the most consistent solver, that outperforms the other approaches on the more challenging test suite by Coelho et al. [6]. While the number of islands is generally a good predictor for the performance, other puzzle features such as the number of solutions or the number of unconnected solution candidates fulfilling Rules 1 to 5 also have a significant influence, especially for the mixed approaches.

We further see that the choice of encoding is important for the performance. With the decision between reachability approach and mixed approach, we face a trade-off between the number of constraints and the number of solver runs. For SAT, the mixed approach performed better overall, while for SMT the reachability approach is clearly better due to the more efficient handling of numeric constraints.

The results of this thesis open several directions for future research. Three aspects appear especially promising: a deeper analysis of structural puzzle properties, a better understanding of puzzle difficulty, and further improvements of the solving approaches.

The evaluation results indicate that structural properties have a significant influence on solver performance. Relevant structural properties include the number of solutions, the number of unconnected solution candidates that satisfy Rules 1 to 5, cycles in the puzzle graph [6] and double bridges in the puzzle [6]. A more detailed analysis of these properties and their influence on solver performance could provide valuable insights.

Analysing puzzle features directly leads to the question of what makes a puzzle difficult. In particular, it is interesting to distinguish between computational difficulty and human-perceived difficulty, and to investigate how closely these align. The performance differences observed between our test suite and the test suite by Coelho et al. [6] suggest that structural properties introduced by the different generation mechanisms play an important role for computational difficulty. It has yet to be determined whether these differences in computational complexity correspond to the human difficulty perception. By understanding

these mechanisms, the generation process can be adapted to produce puzzles with desired difficulty levels.

Finally, the implemented solving approaches could be optimised. In the current implementation, SAT and SMT solvers are used as black-box components. The solvers are repeatedly restarted for finding multiple solutions and in mixed approaches, which leads to a significant amount of redundant computations. Efficiency could therefore be significantly improved by using integrated or incremental SAT- and SMT-solving strategies. The deduction-based HashiSolver can also be further optimised, especially its backtracking algorithm. Stronger pruning strategies and improved branching heuristics could reduce the search space. If uniqueness of the solution is guaranteed, the property of uniqueness could be exploited during solving, since certain decisions could be ruled out if they necessarily lead to multiple solutions. A further promising approach is to combine deduction-based reasoning with SAT or SMT solving. Such a hybrid approach could combine the strengths of both approaches to achieve a better overall performance.

Hashiwokakero has proven to be an interesting study object, due to its NP-completeness and the complexity of the global connectivity constraint. The results of this thesis demonstrate the strengths and limitations of SAT-, SMT-, and deduction-based solvers, and show that performance depends not only on the underlying solving paradigm, but also on the specific encoding strategy. With this, we conclude our journey through Hashiwokakero. We hope the path through computational complexity, solver design and performance analysis has been as satisfying as finally connecting all islands into a valid solution.

**Tools and Reproducibility** DeepL and ChatGPT were used to improve the language of this thesis. All figures and benchmark plots presented in this thesis were created by the author. The full implementation and benchmark materials are available at <https://github.com/hrattenberger/Hashiwokakero>.

## References

- [1] C. Browne. “Deductive search for logic puzzles”. In: *IEEE Conference on Computational Intelligence in Games (CIG), Ontario, Canada*. IEEE, 2013, pp. 1–8. DOI: 10.1109/CIG.2013.6633649.
- [2] G. Kendall, A. J. Parkes, and K. Spoerer. “A Survey of NP-Complete Puzzles”. In: *ICGA J.* 31.1 (2008), pp. 13–34.
- [3] D. Andersson. “Hashiwokakero is NP-complete”. In: *Inf. Process. Lett.* 109.19 (2009), pp. 1145–1146. DOI: 10.1016/J.IPL.2009.07.017.
- [4] S. Golan. “Minesweeper on graphs”. In: *Appl. Math. Comput.* 217.14 (2011), pp. 6616–6623. DOI: 10.1016/J.AMC.2011.01.046.
- [5] S. Ruangwises and T. Itoh. “Physical ZKP for Connected Spanning Subgraph: Applications to Bridges Puzzle and Other Problems”. In: *Unconventional Computation and Natural Computation, UCNC 2021, Espoo, Finland*. Ed. by I. Kostitsyna and P. Orponen. Lecture Notes in Computer Science. Springer, 2021, pp. 149–163. DOI: 10.1007/978-3-030-87993-8\_10.
- [6] L. C. Coelho, G. Laporte, A. Lindbeck, et al. “Benchmark Instances and Branch-and-Cut Algorithm for the Hashiwokakero Puzzle”. In: *CoRR* abs/1905.00973 (2019). URL: <http://arxiv.org/abs/1905.00973>.
- [7] R. F. Malik, R. Efendi, and E. A. Pratiwi. “Solving Hashiwokakero Puzzle Game with Hashi Solving Techniques and Depth First Search”. In: *Bulletin of Electrical Engineering and Informatics* 1.1 (2012), pp. 61–68.
- [8] M. Brain, O. Cliffe, and M. De Vos. “A pragmatic programmer’s guide to answer set programming”. In: *Software Engineering for answer set programming (SEA09)*. 2009.
- [9] P. Prosser and C. Unsworth. “Rooted tree and spanning tree constraints”. In: *17th ECAI workshop on modelling and solving problems with constraints*. 2006.
- [10] H. Lloyd, M. Crossley, M. C. Sinclair, et al. “J-POP: Japanese Puzzles as Optimization Problems”. In: *IEEE Trans. Games* 14.3 (2022), pp. 391–402. DOI: 10.1109/TG.2021.3081817.
- [11] M. Maniprambil, N. Hoehing, J. Kapuriya, et al. “TopoBench: Benchmarking LLMs on Hard Topological Reasoning”. In: *CoRR* abs/2603.12133 (2026). DOI: 10.48550/ARXIV.2603.12133.
- [12] T. A. Sudkamp. *Languages and machines: An Introduction to the Theory of Computer Science*. 3rd ed. Uttar Pradesh, India: Pearson Education, 2008. ISBN: 978-8131714751.

- [13] J. E. Hopcroft, R. Motwani, and J. D. Ullman. *Introduction to automata theory, languages, and computation, 2nd Edition*. Addison-Wesley series in computer science. Addison-Wesley-Longman, 2001. ISBN: 978-0-201-44124-6.
- [14] M. Sipser. *Introduction to the theory of computation, Second Edition*. Boston, Massachusetts: Thomson Course Technology, 2006. ISBN: 0-534-95097-3.
- [15] A. Itai, C. H. Papadimitriou, and J. L. Szwarcfiter. “Hamilton Paths in Grid Graphs”. In: *SIAM J. Comput.* 11.4 (1982), pp. 676–686. DOI: 10.1137/0211056.
- [16] N. Alon and A. Kupavskii. “Two notions of unit distance graphs”. In: *J. Comb. Theory A* 125 (2014), pp. 1–17. DOI: 10.1016/J.JCTA.2014.02.006.
- [17] S. A. Cook. “The Complexity of Theorem-Proving Procedures”. In: *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing, 1971, Ohio, USA*. Ed. by M. A. Harrison, R. B. Banerji, and J. D. Ullman. ACM, 1971, pp. 151–158. DOI: 10.1145/800157.805047.
- [18] D. Westreicher. *Slitherlink Reloaded*. 2011. URL: <http://david-westreicher.github.io/static/papers/ba-thesis.pdf> (visited on 05/07/2025).
- [19] A. Biere, M. Heule, H. van Maaren, et al., eds. *Handbook of Satisfiability*. Vol. 185. Frontiers in Artificial Intelligence and Applications. IOS Press, 2009. ISBN: 978-1-58603-929-5.
- [20] A. Ignatiev, A. Morgado, and J. Marques-Silva. “PySAT: A Python Toolkit for Prototyping with SAT Oracles”. In: *Theory and Applications of Satisfiability Testing - SAT 2018, Oxford, UK*. Ed. by O. Beyersdorff and C. M. Wintersteiger. Lecture Notes in Computer Science. Springer, 2018, pp. 428–437. DOI: 10.1007/978-3-319-94144-8\\_26.
- [21] L. M. de Moura and N. S. Bjørner. “Z3: An Efficient SMT Solver”. In: *Tools and Algorithms for the Construction and Analysis of Systems, TACAS 2008, Budapest, Hungary*. Ed. by C. R. Ramakrishnan and J. Rehof. Lecture Notes in Computer Science. Springer, 2008, pp. 337–340. DOI: 10.1007/978-3-540-78800-3\\_24.
- [22] E. M. Clarke, T. A. Henzinger, H. Veith, et al., eds. *Handbook of Model Checking*. Springer, 2018. ISBN: 978-3-319-10574-1. DOI: 10.1007/978-3-319-10575-8.
- [23] G. Audemard and L. Simon. “On the Glucose SAT Solver”. In: *Int. J. Artif. Intell. Tools* 27.1 (2018), 1840001:1–1840001:25. DOI: 10.1142/S0218213018400018.
- [24] D. Beyer. “Software Verification and Verifiable Witnesses - (Report on SV-COMP 2015)”. In: *Tools and Algorithms for the Construction and Analysis of Systems, TACAS 2015, London, UK*. Ed. by C. Baier and C. Tinelli. Lecture Notes in Computer Science. Springer, 2015, pp. 401–416. DOI: 10.1007/978-3-662-46681-0\\_31.